ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC MÁY TÍNH

PHÂN TÍCH & THIẾT KẾ THUẬT TOÁN

Bài tập lớn: Minh họa thuật toán quay lui qua bài toán Sudoku

---

Sinh viên thực hiện : Lã Xuân Hồng
MSSV : 25410056
Lớp : CS112.F31.LT.TTNT
Giảng viên hướng dẫn : Huỳnh Thị Thanh Thương
Ngày nộp bài : Ngày 12 tháng 10 năm 2026

TP. Hồ Chí Minh, Ngày 12 tháng 10 năm 2026

Mục lục

1 Giới thiệu bài toán **3**

1.1 Lý do chọn bài toán . . . 3
1.2 Nguồn gốc của Sudoku . . . 3
1.3 Ứng dụng thực tế . . . 4

2 Mô tả bài toán **5**

3 Mô hình bài toán **7**

3.1 Input . . . 7
3.2 Output . . . 7
3.3 Điều kiện và ràng buộc . . . 7
3.4 Ví dụ Input và Output . . . 8

4 Thiết kế cấu trúc dữ liệu và thuật toán **10**

4.1 Phân tích đặc trưng và ý tưởng giải . . . 10
4.2 Cấu trúc dữ liệu . . . 11
4.3 Thuật toán quay lui theo mô hình 1 . . . 12
4.4 Phân tích độ phức tạp . . . 13
4.5 Các biến thể đã phát triển thêm . . . 14
4.6 Cấu trúc dữ liệu của phần minh họa từng bước . . . 15

5 Ví dụ minh họa các bước thực hiện **17**

5.1 Chọn đề và cách đọc bảng . . . 17
5.2 Diễn biến . . . 19
5.3 Bảng toàn bộ các bước . . . 22

6 Quá trình thiết kế và xây dựng **25**

6.1 Công cụ và cách làm việc . . . 25
6.2 Chọn đề và lập kế hoạch (3/10) . . . 27
6.3 Thiết kế giao diện bằng Claude Design (3/10 - 4/10) . . . 28
6.4 Chuyển thuật toán sang TypeScript (4/10) . . . 29
6.5 Xây dựng trang Giải Sudoku (4/10) . . . 30

6.6 Góp ý từ một phiên Claude đóng vai người dùng (4/10) 31
6.7 Yêu cầu chính thức và đổi góc nhìn (6/10) 32
6.8 Bộ testcase (6/10) 33
6.9 Trang So sánh (6/10) 34
6.10 Trang Mọi lời giải (6/10) 35
6.11 Quy tắc văn phong và viết lại chữ trên website (6/10) 36
6.12 Đề minh họa và sự cố máy chậm (8/10 - 10/10) 37
6.13 Phần việc của em và phần việc của AI 37

7 Thiết kế thực nghiệm **39**
7.1 Cách tạo bộ testcase 39
7.2 Các nhóm testcase 40
7.3 Cách bảo đảm độ phủ 41
7.4 Đáp án lấy từ đâu 42

8 Đánh giá **43**
8.1 Kết quả trên bộ testcase 43
8.2 Thời gian thực hiện 43
8.3 So sánh các cách giải 44
8.4 Kiểm thử tự động 45
8.5 Tính trực quan của giao diện 45
8.6 Hạn chế và hướng phát triển 46

Chương 1

Giới thiệu bài toán

1.1 Lý do chọn bài toán

Em phân vân giữa hai bài: tìm đường ngắn nhất bằng Dijkstra và giải Sudoku. Em thích cả hai. Khi em hỏi ý kiến Claude Code, Claude nhận xét cô chưa dạy Dijkstra trên lớp, chỉ nhắc tới như một bài của khóa trước. Em chọn Sudoku vì ba lý do:

- **Em đã tự giải bài này.** Trong một bài thực hành trên Wecode, em đã cài đặt quay lui theo mô hình 1 cho Sudoku. Em hiểu thuật toán từ bên trong, nên có thể tự kiểm từng bước mà AI làm.
- **Thuật toán nhìn thấy được.** Ba động tác thử số, loại số trùng và lùi về ô trước đều hiện ngay trên lưới. Yêu cầu của bài tập lớn là cho người dùng theo dõi từng bước, và Sudoku cho thấy điều đó rõ hơn một đồ thị.
- **Có chỗ để phát triển.** Cùng một bài toán, em so sánh được nhiều cấu trúc dữ liệu, nhiều thứ tự chọn ô, và chế độ tìm mọi lời giải đúng nguyên văn mô hình 1.

1.2 Nguồn gốc của Sudoku

Trò chơi lưới số dạng hiện nay do Howard Garns soạn, xuất hiện lần đầu năm 1979 trên tạp chí đố vui của Dell ở Mỹ với tên "Number Place". Năm 1984, nhà xuất bản Nikoli đưa trò chơi sang Nhật và đặt tên Sudoku, rút gọn từ một câu tiếng Nhật nghĩa là "mỗi chữ số chỉ được xuất hiện một lần". Từ năm 2004, khi báo The Times ở Anh đăng Sudoku hằng ngày, trò chơi lan ra khắp thế giới [5].

Luật chơi chỉ có một câu: điền lưới 9×9 sao cho mỗi hàng, mỗi cột và mỗi khối 3×3 có đủ các số từ 1 đến 9. Đằng sau câu đó là những con số lớn. Có tổng cộng 6.670.903.752.021.072.936.960 lưới đầy hợp lệ, theo phép tính của Felgenhauer và Jarvis năm 2005. Một đề có lời giải duy nhất cần ít nhất 17 số cho sẵn, McGuire và cộng sự chứng minh năm 2012 [6]. Bộ testcase của em có một đề 17 số lấy từ danh sách của Gordon Royle.

1.3 Ứng dụng thực tế

Sudoku là một bài toán thỏa mãn ràng buộc: có 81 biến (các ô), mỗi biến nhận giá trị trong miền $\{1, \ldots, 9\}$, và các ràng buộc "khác nhau" trong từng hàng, cột, khối. Cùng khung này xuất hiện trong nhiều bài toán thật:

- **Xếp lịch thi.** Hai môn có chung sinh viên không được xếp cùng ca, giống hai ô cùng hàng không được cùng số.
- **Xếp thời khóa biểu.** Một giáo viên không dạy hai lớp cùng tiết, một phòng không chứa hai lớp cùng tiết.
- **Tô màu đồ thị.** Mỗi ô là một đỉnh. Hai ô cùng hàng, cột hoặc khối nối với nhau bằng một cạnh. Mỗi đỉnh có đúng 20 đỉnh kề: 8 ô cùng hàng, 8 ô cùng cột, 4 ô còn lại cùng khối. Giải Sudoku chính là tô đồ thị 81 đỉnh, 810 cạnh bằng 9 màu sao cho hai đỉnh kề khác màu, với một số đỉnh đã tô sẵn.

Quay lui cũng là cách giải chung cho các bài cùng loại trong slide của cô: 8 quân hậu, tổng tập con, chu trình Hamilton, tô màu đồ thị [1]. Với Sudoku tổng quát cỡ $n^2 \times n^2$, bài toán thuộc lớp NP-đầy đủ [6], nên chưa ai biết cách giải luôn nhanh cho mọi đề. Hiểu quay lui chạy thế nào, và vì sao có đề làm thuật toán chậm, là bước đầu để hiểu các bài toán khó này.

Bản thân website là một ứng dụng dạy học. Người mới học thuật toán nhìn được thuật toán thử, loại và lùi lại trên từng ô, thay vì chỉ đọc mã giả.

Chương 2

Mô tả bài toán

Sudoku là trò chơi điền số trên lưới 9×9. Lưới chia thành 9 khối 3×3 bằng các đường kẻ đậm. Một lưới Sudoku hợp lệ khi mỗi hàng, mỗi cột và mỗi khối đều chứa đủ 9 số từ 1 đến 9, mỗi số đúng một lần.

Cho một lưới trong đó một số ô đã điền sẵn, các ô còn lại để trống. Hãy điền số vào các ô trống để được một lưới Sudoku hợp lệ, giữ nguyên các số đã cho.

Input

Gồm 9 dòng, mỗi dòng 9 ký tự cách nhau bởi dấu cách. Mỗi ký tự là một số từ 1 đến 9 (ô đã điền) hoặc chữ `x` (ô trống). Số ô trống từ 0 đến 81.

Output

- Nếu điền được: 9 dòng, mỗi dòng 9 số cách nhau bởi dấu cách, là lưới đã điền kín và hợp lệ. Nếu có nhiều cách điền, xuất cách đầu tiên tìm được khi xét ô trống theo thứ tự hàng rồi cột và thử số từ nhỏ đến lớn.
- Nếu đề có hai ô cùng số trong cùng một hàng, cột hoặc khối: thông báo đề không hợp lệ, kèm số trùng và vị trí.
- Nếu không có cách điền nào: thông báo đề không có lời giải.

Ví dụ

Input

```
5 3 4 6 7 X 9 X X
6 7 X X 9 5 3 4 8
X 9 8 3 4 X 5 6 7
8 X 9 7 6 X X 2 3
4 X 6 X 5 3 7 9 1
7 1 3 9 X 4 8 5 6
9 6 1 5 3 7 2 X 4
2 8 7 4 1 9 6 3 5
X 4 5 X 8 6 1 7 9
```

Output

```
5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9
```

Yêu cầu

Giải bằng kỹ thuật quay lui theo mô hình 1. Chương trình phải cho người dùng nhập đề, theo dõi từng bước thuật toán thực hiện, và xem kết quả.

Chương 3

Mô hình bài toán

3.1 Input

- A: ma trận 9×9. $A[r][c] \in \{1, \ldots, 9\} \cup \{\text{x}\}$ với $1 \leq r, c \leq 9$. $A[r][c]$ là số đề cho ở ô hàng r, cột c; x nghĩa là ô đó trống.
- Website nhận thêm hai cách viết đề: 9 dòng viết liền không dấu cách, và một chuỗi 81 ký tự trong đó dấu chấm là ô trống.

3.2 Output

Một trong ba kết quả:

- B: ma trận 9×9, $B[r][c] \in \{1, \ldots, 9\}$, là lưới đã điền kín và hợp lệ.
- Thông báo đề không hợp lệ, kèm số trùng, nằm ở hàng, cột hay khối nào, và các ô trùng.
- Thông báo đề không có lời giải.

3.3 Điều kiện và ràng buộc

- Số ô trống n thỏa $0 \leq n \leq 81$.
- Khối thứ b, với $1 \leq b \leq 9$, gồm các ô có hàng từ $3\lfloor (b-1)/3 \rfloor + 1$ đến $3\lfloor (b-1)/3 \rfloor + 3$ và cột từ $3\left((b-1) \bmod 3\right) + 1$ đến $3\left((b-1) \bmod 3\right) + 3$.
- **Đề hợp lệ** khi không có hai ô đã điền nào cùng số mà nằm chung hàng, chung cột hoặc chung khối.
- **Giữ nguyên số đề cho:** $B[r][c] = A[r][c]$ với mọi ô có $A[r][c] \neq \text{x}$.

- **Lưới hợp lệ:** mỗi hàng r có $\{B[r][1], \ldots, B[r][9]\} = \{1, \ldots, 9\}$. Mỗi cột và mỗi khối cũng vậy.
- **Nhiều lời giải:** chọn lời giải đầu tiên tìm được khi xét ô trống theo thứ tự hàng rồi cột, thử số từ 1 đến 9. Quy ước này làm đầu ra xác định, nên so được với bản cài đặt tham chiếu từng ký tự.

3.4 Ví dụ Input và Output

Ví dụ 1: đề có lời giải duy nhất

Đề số 13 ở Chương 2. Chương 5 đi qua toàn bộ các bước giải đề này.

Ví dụ 2: đề nhiều lời giải

Đề "Nhiều lời giải" có 44 ô trống và 18 lời giải. Output là lời giải đầu tiên, thuật toán tìm ra sau 827 lần thử.

Input

```
5 3 X 6 X X 9 1 2
X X X X 9 5 3 4 X
1 9 X X X 2 X 6 X
X X X 7 6 1 X 2 X
X X 6 X X 3 7 X X
X 1 X 9 2 X 8 X X
X X 1 X X X X 8 4
X X X 4 1 X X 3 5
X 4 X 2 X 6 1 7 X
```

Output

```
5 3 4 6 7 8 9 1 2
2 6 7 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
6 7 1 5 3 9 2 8 4
9 8 2 4 1 7 6 3 5
3 4 5 2 8 6 1 7 9
```

Ví dụ 3: đề không hợp lệ

Hai số 7 cùng nằm trên hàng 4.

Input

```
X X X X X X X X X
X X X X X X X X X
X X X X X X X X X
7 X X X X X X X 7
X X X X X X X X X
X X X X X X X X X
X X X X X X X X X
X X X X X X X X X
X X X X X X X X X
```

Output

Đề không hợp lệ: số 7 xuất hiện 2 lần ở hàng 4 (cột 1 và cột 9).

Ví dụ 4: đề không có lời giải

Hàng 1 đã có các số từ 2 đến 9, cột 1 đã có số 1, nên ô $(1, 1)$ không nhận được số nào.

Input

```
X 2 3 4 5 6 7 8 9
1 X X X X X X X X
X X X X X X X X X
X X X X X X X X X
X X X X X X X X X
X X X X X X X X X
X X X X X X X X X
X X X X X X X X X
X X X X X X X X X
```

Output

Đề này không có lời giải. Ở ô $(1, 1)$, cả 9 số đều trùng: số 1 đã có ở cột 1, các số từ 2 đến 9 đã có ở hàng 1.

Chương 4

Thiết kế cấu trúc dữ liệu và thuật toán

Chiến lược giải là **quay lui** (Backtracking) theo **mô hình 1** trong slide của cô [1]. Chương này đi theo thứ tự cô hướng dẫn cho câu thiết kế: phân tích đặc trưng để chọn chiến lược, mô hình hóa cấu trúc lời giải, chọn cấu trúc dữ liệu, viết mã giả, rồi tính độ phức tạp.

4.1 Phân tích đặc trưng và ý tưởng giải

Bài toán có đủ hai dấu hiệu của quay lui:

- **Lời giải gồm nhiều bước.** Mỗi ô trống là một bước. Đề Arto Inkala có 60 ô trống, tức 60 bước.
- **Mỗi bước có nhiều lựa chọn.** Mỗi ô trống có 9 số để chọn, từ 1 đến 9.

Không có tiêu chí "tốt nhất" nào cần tối ưu, chỉ cần một cách điền đúng luật. Vậy đây là bài toán quyết định, hợp với quay lui hơn nhánh cận.

Mô hình hóa cấu trúc lời giải

- Đánh số các ô trống lần lượt từ trái sang phải, từ trên xuống dưới: ô trống thứ $1, 2, \ldots, n$.
- Lời giải là vectơ $S = (S_1, S_2, \ldots, S_n)$. $S_i \in \{1, \ldots, 9\}$ là số điền vào ô trống thứ i.
- n là số thành phần của lời giải, bằng số ô trống. $k = 9$ là số khả năng mà mỗi S_i có thể chọn.
- Thứ tự các thành phần có nghĩa: S_i gắn với đúng một ô cố định trên lưới.
- **Khả năng $S_i = j$ chấp nhận được** khi số j chưa có ở hàng, ở cột và ở khối 3×3 chứa ô trống thứ i.

Ý tưởng

Thuật toán điền lần lượt từng ô trống. Ở ô trống thứ i, thuật toán thử $j = 1, 2, \ldots, 9$. Số nào trùng hàng, cột hoặc khối thì bỏ ngay, nên cả nhánh phía sau số đó không bao giờ phải thử. Số nào chấp nhận được thì điền vào rồi sang ô trống thứ $i + 1$. Ô nào thử hết 9 số mà không số nào đi tiếp được thì xóa số vừa điền và lùi về ô trước, thử số kế tiếp ở đó.

Đề số 13 trong Chương 5 cho thấy ý tưởng này trên lưới thật. Ở lần thử 1, ô $(1, 6)$ nhận số 1. Ô kế tiếp $(1, 8)$ thử cả 9 số đều trùng, nên ở lần thử 10 thuật toán lùi về $(1, 6)$, xóa số 1 và thử số 2.

4.2 Cấu trúc dữ liệu

Bảng 4.1 liệt kê các biến của thuật toán, kiểu, miền giá trị và vai trò.

Bảng 4.1: Cấu trúc dữ liệu của thuật toán quay lui

Biến	Kiểu, miền giá trị	Vai trò
`grid`	Ma trận 9×9 số nguyên trong $[0, 9]$	Trạng thái lưới hiện tại. Số 0 là ô trống
`emptyCells`	Mảng n cặp (hàng, cột)	Ô trống thứ i nằm ở đâu, theo thứ tự hàng rồi cột
n	Số nguyên, $0 \le n \le 81$	Số ô trống, cũng là số thành phần của lời giải
$S[1..n]$	Mảng n số nguyên trong $[1, 9]$	Vectơ lời giải $S = (S_1, \ldots, S_n)$
`rowUsed`	Ma trận 9×9 giá trị đúng/sai	`rowUsed[r][d]` đúng khi hàng r đã có số d
`columnUsed`	Ma trận 9×9 giá trị đúng/sai	`columnUsed[c][d]` đúng khi cột c đã có số d
`boxUsed`	Ma trận 9×9 giá trị đúng/sai	`boxUsed[b][d]` đúng khi khối b đã có số d

Khối chứa ô (r, c), với hàng và cột đánh số từ 1, là

$$b(r, c) = 3 \left\lfloor \frac{r-1}{3} \right\rfloor + \left\lfloor \frac{c-1}{3} \right\rfloor + 1.$$

Ba mảng đánh dấu là chỗ quyết định tốc độ. Không có chúng, mỗi lần kiểm tra số j phải đọc lại cả hàng, cả cột và cả khối, tới 27 ô. Có chúng, chỉ cần đọc ba giá trị `rowUsed[r][j]`, `columnUsed[c][j]`, `boxUsed[b][j]`, và mỗi lần điền hay xóa một số chỉ cần sửa ba giá trị. Cái giá là $3 \times 9 \times 9 = 243$ giá trị đúng/sai, một lượng bộ nhớ cố định không phụ thuộc vào đề.

4.3 Thuật toán quay lui theo mô hình 1

Mô hình 1 làm một việc lặp đi lặp lại. Ở bước i, thử lần lượt mọi khả năng j. Khả năng nào chấp nhận được thì ghi nhận rồi đi tiếp sang bước $i+1$, quay về thì trả lại trạng thái cũ. Mã giả trong slide của cô:

```
try (i) =
{     for (j = 1 --> k)   // k: số khả năng mà S_i có thể chọn
           if (khả năng j chấp nhận được)
           {   Xác định S_i theo j;
               Ghi nhận trạng thái mới của bài toán (nếu có);
               if (i < n)
                     try (i+1); // tiến hành bước i+1 để xác định tiếp S_{i+1}
               else
                     Xuất lời giải S = (S_1, S_2, …, S_n);
               Trả lại trạng thái cũ của bài toán;
           }
}
```

Áp dụng cho Sudoku, thuật toán có ba bước.

Bước 1. Chuẩn bị. Đọc lưới. Với mỗi ô (r, c) có sẵn số d, bật cờ của số d ở hàng r, cột c và khối $b(r, c)$ trong ba mảng đánh dấu. Với mỗi ô trống, thêm (r, c) vào cuối `emptyCells`. Đặt n bằng số ô trống.

Bước 2. Kiểm tra đề. Nếu một số xuất hiện hai lần trong cùng hàng, cột hoặc khối thì báo đề không hợp lệ và dừng. Nếu $n = 0$ thì lưới đã đầy, không cần giải.

Bước 3. Gọi `try(1)`. Nếu `try(1)` chạy xong mà chưa xuất lời giải nào thì đề vô nghiệm.

```
try (i) =
{     (r, c) = emptyCells[i];
      for (j = 1 --> 9)
           if (!rowUsed[r][j] && !columnUsed[c][j] && !boxUsed[b(r,c)][j])
           {   S[i] = j;
               grid[r][c] = j;
               rowUsed[r][j] = columnUsed[c][j] = boxUsed[b(r,c)][j] = true;
               if (i < n)
                     try (i+1);
               else
                     Xuất lời giải grid và dừng;
               grid[r][c] = 0;
               rowUsed[r][j] = columnUsed[c][j] = boxUsed[b(r,c)][j] = false;
           }
}
```

Bảng 4.2 đối chiếu từng dòng mô hình 1 với bản dành cho Sudoku.

Bảng 4.2: Đối chiếu mô hình 1 với bản dành cho Sudoku

Mô hình 1	**Với Sudoku**
`for (j = 1 --> k)`	Thử j từ 1 đến 9, vì $k = 9$
`if (khả năng j chấp nhận được)`	Hàng, cột và khối của ô trống thứ i đều chưa có số j, kiểm tra qua ba mảng đánh dấu
`Xác định` S_i `theo j`	`S[i] = j`
`Ghi nhận trạng thái mới`	Ghi j vào lưới, bật ba cờ đánh dấu
`if (i < n) try(i+1)`	Sang ô trống kế tiếp
`Xuất lời giải`	Xuất lưới đã điền kín
`Trả lại trạng thái cũ`	Xóa j khỏi lưới, tắt ba cờ đánh dấu

Bản dành cho Sudoku khác mô hình 1 ở một chỗ: bản này dừng ngay sau lời giải đầu tiên. Đề bài chỉ cần một cách điền, nên đi tiếp là thừa. Trang "Mọi lời giải" trên website chạy đúng nguyên văn mô hình 1, không dừng, để đếm đề có bao nhiêu lời giải (mục 4.5).

4.4 Phân tích độ phức tạp

Gọi $T(n)$ là chi phí khi còn n ô trống. Mỗi lần gọi `try` thử 9 số. Mỗi lần kiểm tra chỉ đọc ba mảng đánh dấu, mỗi lần ghi nhận hay trả lại chỉ sửa ba giá trị, nên phần việc ngoài các lời gọi đệ quy tốn một hằng số c_2. Mỗi số chấp nhận được mở ra một bài toán con còn $n-1$ ô trống. Trường hợp xấu nhất là cả 9 số đều chấp nhận được:

$$T(n) = 9\,T(n-1) + c_2, \qquad T(0) = c_1.$$

Giải bằng phương pháp truy hồi:

$$\begin{aligned} T(n) &= 9\,T(n-1) + c_2 \\ &= 9\,[9\,T(n-2) + c_2] + c_2 = 9^2\,T(n-2) + 9c_2 + c_2 \\ &= 9^2\,[9\,T(n-3) + c_2] + 9c_2 + c_2 = 9^3\,T(n-3) + (9^2 + 9 + 1)\,c_2 \\ &= \dots \\ &= 9^i\,T(n-i) + c_2 \sum_{t=0}^{i-1} 9^t \qquad (*) \end{aligned}$$

Quá trình dừng lại khi $n - i = 0 \iff i = n$. Ta có $\sum_{t=0}^{i-1} 9^t = \dfrac{9^i - 1}{9 - 1} = \dfrac{9^i - 1}{8}$. Thay

$i = n$ vào $(*)$:

$$T(n) = 9^n\,T(0) + c_2\,\frac{9^n - 1}{8} = \left(c_1 + \frac{c_2}{8}\right) 9^n - \frac{c_2}{8} \in \mathcal{O}(9^n)\,.$$

Con số 9^n lớn tới mức nào? Đề Arto Inkala có $n = 60$, nên cận trên là 9^{60}, một số có 58 chữ số. Thuật toán thực tế chỉ cần 445.778 lần thử cho đề này. Khoảng cách đó đến từ việc bỏ sớm: số nào trùng thì thuật toán cắt luôn cả nhánh phía sau. Cận $\mathcal{O}(9^n)$ vẫn có thật. Đề "chống quay lui" trong Chương 7 cố ý đặt số đúng ở cuối mỗi lượt thử, nên cần tới 622.577.597 lần thử với chỉ 64 ô trống.

Về bộ nhớ, ba mảng đánh dấu và lưới có kích thước cố định. Vectơ S, danh sách ô trống và độ sâu đệ quy cùng tỉ lệ với n. Tổng bộ nhớ là $\mathcal{O}(n)$, với $n \le 81$.

4.5 Các biến thể đã phát triển thêm

Ngoài bản chính, website có ba biến thể để so sánh. Cả ba vẫn giữ khung mô hình 1.

Ba cách kiểm tra "chấp nhận được"

Thứ tự xét ô giữ nguyên, chỉ đổi cấu trúc dữ liệu dùng để kiểm tra:

- **Quét lưới.** Không dùng mảng phụ. Mỗi lần kiểm tra đọc lại hàng, cột và khối, tới 27 ô.
- **Ba mảng đánh dấu.** Cách của bản chính, đọc tối đa 3 giá trị.
- **Bitmask.** Mỗi hàng, cột, khối gói vào một số nguyên, bit thứ d bật khi số d đã có. Kiểm tra bằng một phép OR trên ba số rồi một phép AND với bit thứ j.

Ba cách cho đúng cùng cây tìm kiếm, nên cùng số lần thử trên mọi đề. Cái khác là hằng số c_2 trong $T(n) = 9\,T(n-1) + c_2$. Đo trên đề Arto Inkala, quét lưới mất khoảng 9,5 ms, hai cách còn lại khoảng 4 đến 5 ms (Chương 8).

Chọn ô ít khả năng nhất trước

Thay vì lấy ô trống kế tiếp theo hàng rồi cột, trước mỗi bước thuật toán đếm số giá trị còn chấp nhận được của mọi ô trống chưa điền, rồi chọn ô có ít nhất. Hai ô bằng nhau thì lấy ô đứng trước theo hàng rồi cột. Cách này thường gọi là MRV (minimum remaining values) [3, 4]. S_i lúc này là số điền vào ô được chọn ở bước i, không còn gắn với một ô cố định.

- **Cây nhỏ đi.** Đề Arto Inkala giảm từ 445.778 xuống 90.665 lần thử. Đề chống quay lui giảm từ 622.577.597 xuống 407.156.
- **Mỗi bước đắt hơn.** Việc chọn ô tốn $\mathcal{O}(9n)$ phép kiểm tra thay cho một hằng số. Trên đề Inkala, tổng thời gian vì thế lại dài hơn bản chính.

- **Cận trên không đổi.** Trường hợp xấu nhất vẫn là $\mathcal{O}(9^n)$.

Tìm mọi lời giải

Biến thể này chạy đúng nguyên văn mô hình 1: sau khi xuất một lời giải, thuật toán trả lại trạng thái cũ và thử tiếp, tới khi hết mọi khả năng. Thuật toán luôn đi hết cây tìm kiếm, nên số lần thử lớn hơn hẳn bản dừng sớm. Với đề Arto Inkala, tìm lời giải đầu tiên cần 445.778 lần thử, còn đi hết để biết chắc chỉ có một lời giải cần 18.619.020 lần. Biến thể này cho biết đề mẫu "Vừa" có 4 lời giải và đề "Nhiều lời giải" có 18.

4.6 Cấu trúc dữ liệu của phần minh họa từng bước

Phần minh họa cần dừng thuật toán ở bất kỳ bước nào rồi chạy tiếp. Hàm đệ quy bình thường chạy một mạch nên không làm được việc đó. Website viết `try` thành một *generator*: hàm có thể tạm dừng ở giữa và tiếp tục từ đúng chỗ đó. Ở mỗi dòng của mô hình 1, generator phát ra một sự kiện (Bảng 4.3).

Bảng 4.3: Sự kiện generator phát ra và cách giao diện hiển thị

Sự kiện	**Dòng mô hình 1**	**Hiển thị**
`acceptCandidate`	Khả năng j chấp nhận được	Ô nền vàng, mã giả gắn nhãn "đúng"
`rejectCandidate`	Khả năng j không chấp nhận được	Số gạch ngang, ô gây trùng có vòng đỏ, câu nêu lý do
`recordState`	Ghi nhận trạng thái mới	Ô giữ số j, S_i điền vào bảng S
`descend`	`try(i+1)`	Ô trống kế tiếp thành ô đang xét
`restoreState`	Trả lại trạng thái cũ	Ô xóa số, nền tím viền nét đứt
`solutionFound`	Xuất lời giải (bản dừng sớm)	Các ô thuật toán điền chuyển màu xanh ngọc
`noSolution`	`try(1)` kết thúc không lời giải	Báo đề vô nghiệm
`solutionListed`	Xuất lời giải (bản tìm hết)	Thêm lời giải vào bộ sưu tập, chạy tiếp
`searchExhausted`	`try(1)` kết thúc (bản tìm hết)	Báo tổng số lời giải

Ba cấu trúc nữa giữ phần minh họa chạy mượt:

- **Vòng phát lại theo khung hình.** Mỗi khung hình, khoảng 60 lần một giây, lấy một loạt sự kiện theo tốc độ người xem chọn, áp vào trạng thái, rồi mới vẽ lại một lần. Vẽ lại sau từng sự kiện thì ở tốc độ cao trình duyệt sẽ treo.

- **Lịch sử 100 bước gần nhất.** Website không lưu bản sao lưới ở mỗi bước, chỉ lưu sự kiện và vài giá trị đi kèm. Khi người xem chọn một bước cũ, website hoàn tác ngược từ trạng thái hiện tại về bước đó.
- **Bộ đếm** lần thử, ghi nhận, trả lại, cập nhật theo từng sự kiện.

Generator có cái giá của nó. Trên đề Arto Inkala, bản đệ quy thường chạy hết 3,7 ms, bản generator mất 623 ms. Mỗi sự kiện phải đi ngược qua từng tầng đệ quy, có lúc tới 60 tầng. Phần so sánh tốc độ ở trang So sánh vì thế dùng bản đệ quy thường, để chi phí của generator không che mất khác biệt giữa các cách giải.

Chương 5

Ví dụ minh họa các bước thực hiện

Ví dụ của chương này là đề số 13, tên "Ngắn, có quay lui" trong danh sách đề mẫu của website. Đề có 16 ô trống và đúng một lời giải. Thuật toán cần 79 lần thử, ghi nhận 19 lần và quay lui 3 lần, trong đó có một lần lùi dây chuyền qua hai ô. Toàn bộ 83 bước nằm trong Bảng 5.2 ở cuối chương.

5.1 Chọn đề và cách đọc bảng

Em cần một đề đủ ngắn để in trọn từng bước, mà vẫn có quay lui thật. Đề Arto Inkala cần 445.778 lần thử, quá dài. Đề "Dễ" chỉ cần 27 lần thử nhưng không quay lui lần nào. Script `checks/find_illustration_puzzle.py` xóa ngẫu nhiên 10 đến 16 ô từ một lưới đầy, rồi giữ lại các đề thỏa: đúng một lời giải, 50 đến 90 lần thử, 1 đến 3 lần quay lui, có đủ ba kiểu bị loại vì hàng, cột và khối. Trong 12 đề tìm được, em chọn đề này vì cả ba lần quay lui nằm gọn trong 35 lần thử đầu, ngay trên hàng 1.

5	**3**	**4**	**6**	**7**		**9**		
6	**7**			**9**	**5**	**3**	**4**	**8**
	9	**8**	**3**	**4**		**5**	**6**	**7**
8		**9**	**7**	**6**			**2**	**3**
4		**6**		**5**	**3**	**7**	**9**	**1**
7	**1**	**3**	**9**		**4**	**8**	**5**	**6**
9	**6**	**1**	**5**	**3**	**7**	**2**		**4**
2	**8**	**7**	**4**	**1**	**9**	**6**	**3**	**5**
	4	**5**		**8**	**6**	**1**	**7**	**9**

(a) Đề: 16 ô trống

5	**3**	**4**	**6**	**7**	8	**9**	1	2
6	**7**	2	1	**9**	**5**	**3**	**4**	**8**
1	**9**	**8**	**3**	**4**	2	**5**	**6**	**7**
8	5	**9**	**7**	**6**	1	4	**2**	**3**
4	2	**6**	8	**5**	**3**	**7**	**9**	**1**
7	**1**	**3**	**9**	2	**4**	**8**	**5**	**6**
9	**6**	**1**	**5**	**3**	**7**	**2**	8	**4**
2	**8**	**7**	**4**	**1**	**9**	**6**	**3**	**5**
3	**4**	**5**	2	**8**	**6**	**1**	**7**	**9**

(b) Lời giải: số in xanh là số thuật toán điền

Hình 5.1: Đề số 13 và lời giải

Bảng 5.1 đánh số 16 ô trống theo thứ tự hàng rồi cột. S_i là số điền vào ô trống thứ i.

Bảng 5.1: Vị trí của 16 ô trống

i	1	2	3	4	5	6	7	8
Ô	(1, 6)	(1, 8)	(1, 9)	(2, 3)	(2, 4)	(3, 1)	(3, 6)	(4, 2)
i	9	10	11	12	13	14	15	16
Ô	(4, 6)	(4, 7)	(5, 2)	(5, 4)	(6, 5)	(7, 8)	(9, 1)	(9, 4)

Trạng thái dữ liệu gồm lưới `grid`, vectơ S và ba mảng đánh dấu `rowUsed`, `columnUsed`, `boxUsed` (Chương 4). Mỗi dòng của Bảng 5.2 là một bước thuật toán:

- **Bấm**: số lần bấm nút Từng bước trên trang Giải Sudoku để tới đúng trạng thái đó. Mỗi lần bấm, trang đi một sự kiện, kể cả các sự kiện không có dòng riêng trong bảng như "ghi nhận" và "sang ô kế". Nhờ cột này, người đọc mở website và kiểm lại được bất kỳ dòng nào.
- **Lần thử**, i, **Ô**, j: bộ đếm lần thử, ô trống thứ mấy, nằm ở đâu, và số đang thử.
- **Thao tác** và **Lý do**: "Chấp nhận" là số j chấp nhận được, thuật toán ghi nhận rồi sang ô trống kế tiếp. "Bị loại" kèm hàng, cột hoặc khối bị trùng và ô đang giữ số đó. "Trả lại" là trả lại trạng thái cũ vì ô phía sau đã hết số để thử.
- S **sau bước**: vectơ $S_1 \dots S_{16}$ sau khi bước đó xong, dấu chấm là thành phần chưa có giá trị.

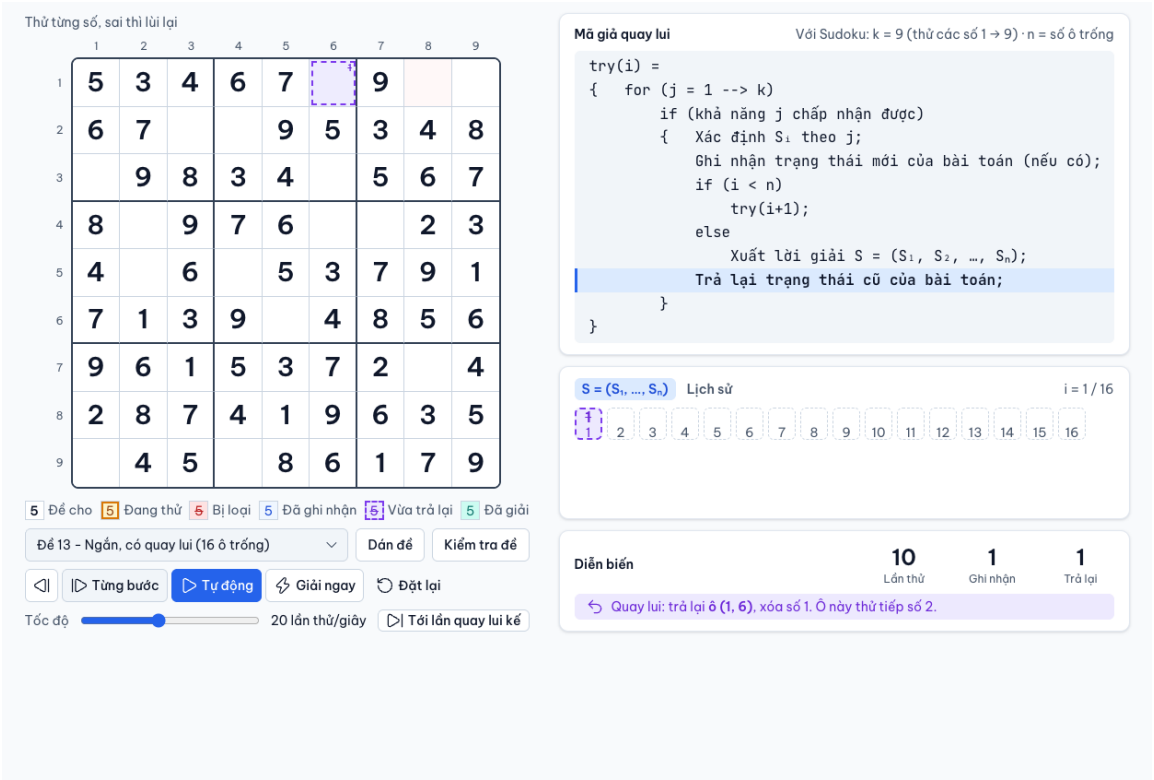
5.2 Diễn biến

Hàng 1 đã có các số 5, 3, 4, 6, 7, 9, nên ba ô trống $(1,6)$, $(1,8)$, $(1,9)$ phải nhận đúng ba số 1, 2, 8. Số 8 đã nằm ở ô $(2,9)$, cùng cột 9 với ô $(1,9)$ và cùng khối 3 với cả hai ô $(1,8)$, $(1,9)$. Vậy chỉ ô $(1,6)$ nhận được số 8. Người giải bằng tay thấy ngay điều này. Thuật toán thì thử từ nhỏ lên lớn, nên phải đi vào ngõ cụt hai lần mới tới số 8.

Giai đoạn 1: lần thử 1 đến 10, quay lui một ô

Ở lần thử 1, ô $(1,6)$ thử số 1. Hàng 1, cột 6 và khối 2 đều chưa có số 1, nên số 1 chấp nhận được. Thuật toán ghi nhận: $S_1 = 1$, `grid[1][6] = 1`, bật `rowUsed[1][1]`, `columnUsed[6][1]`, `boxUsed[2][1]`. Rồi sang ô trống thứ 2.

Ô $(1,8)$ thử 9 số và cả 9 đều trùng. Số 1 trùng ngay với số vừa điền ở ô $(1,6)$. Số 2 trùng cột 8, số 8 trùng khối 3, các số còn lại trùng hàng 1. Ô $(1,8)$ hết số, nên ở lần thử 10 (lần bấm 13) thuật toán trả lại trạng thái cũ của ô $(1,6)$: xóa số 1 khỏi lưới, tắt ba cờ vừa bật, S_1 trở về trống (Hình 5.2).

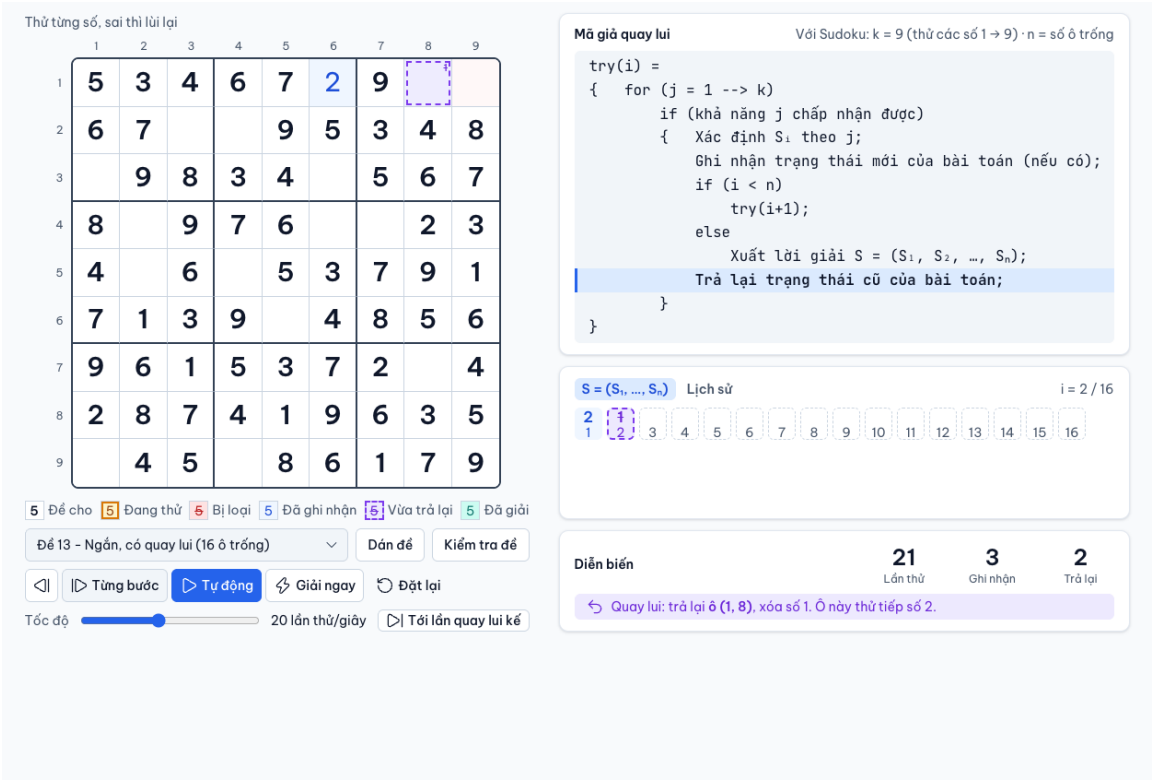


Hình 5.2: Lần bấm 13: thuật toán vừa trả lại ô (1, 6), xóa số 1. Bộ đếm: 10 lần thử, 1 ghi nhận, 1 trả lại

Giai đoạn 2: lần thử 11 đến 29, quay lui dây chuyền

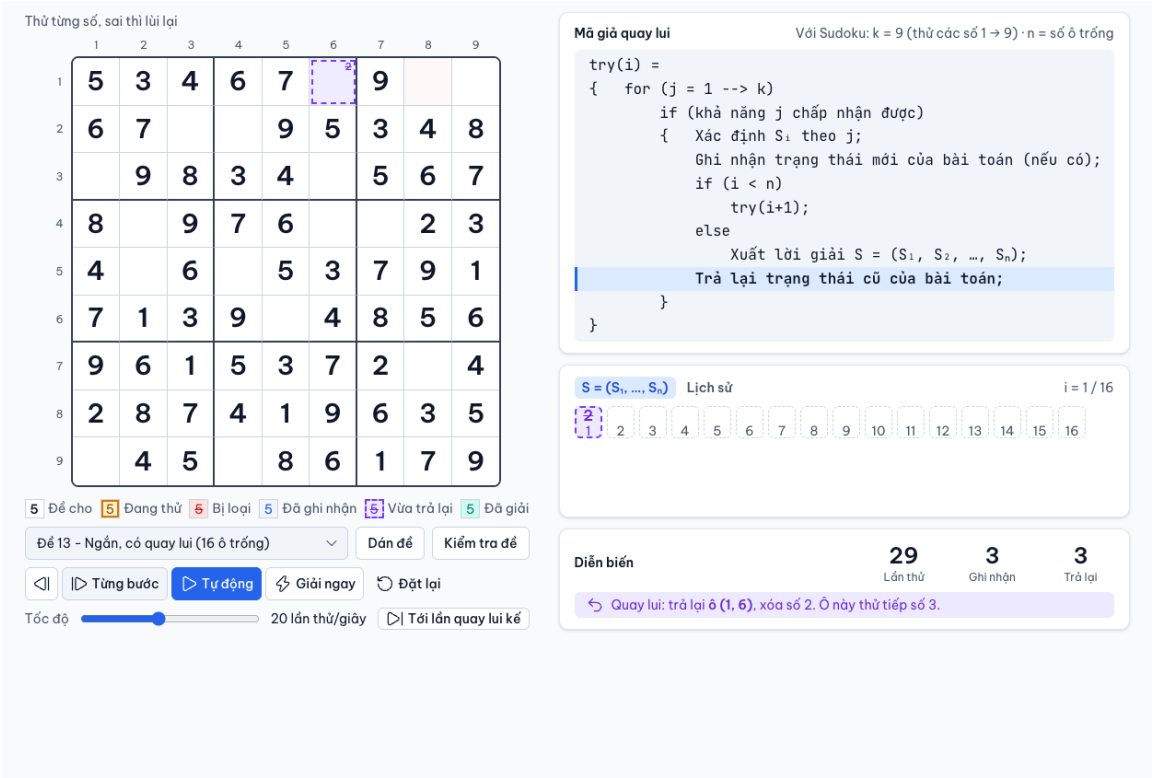
Ô $(1,6)$ thử tiếp số 2 và chấp nhận được (lần thử 11). Lần này ô $(1,8)$ nhận được số 1, vì số 1 không còn ở hàng 1 nữa (lần thử 12). Ô trống thứ 3, ô $(1,9)$, thử 9 số đều trùng: số 1 trùng với

ô $(1, 8)$ vừa điền, số 2 trùng với ô $(1, 6)$, số 8 trùng cột 9, các số còn lại trùng hàng 1. Ở lần thử 21 (lần bấm 29), thuật toán trả lại ô $(1, 8)$, xóa số 1 (Hình 5.3).



Hình 5.3: Lần bấm 29: thuật toán vừa trả lại ô (1, 8), xóa số 1. Bộ đếm: 21 lần thử, 3 ghi nhận, 2 trả lại

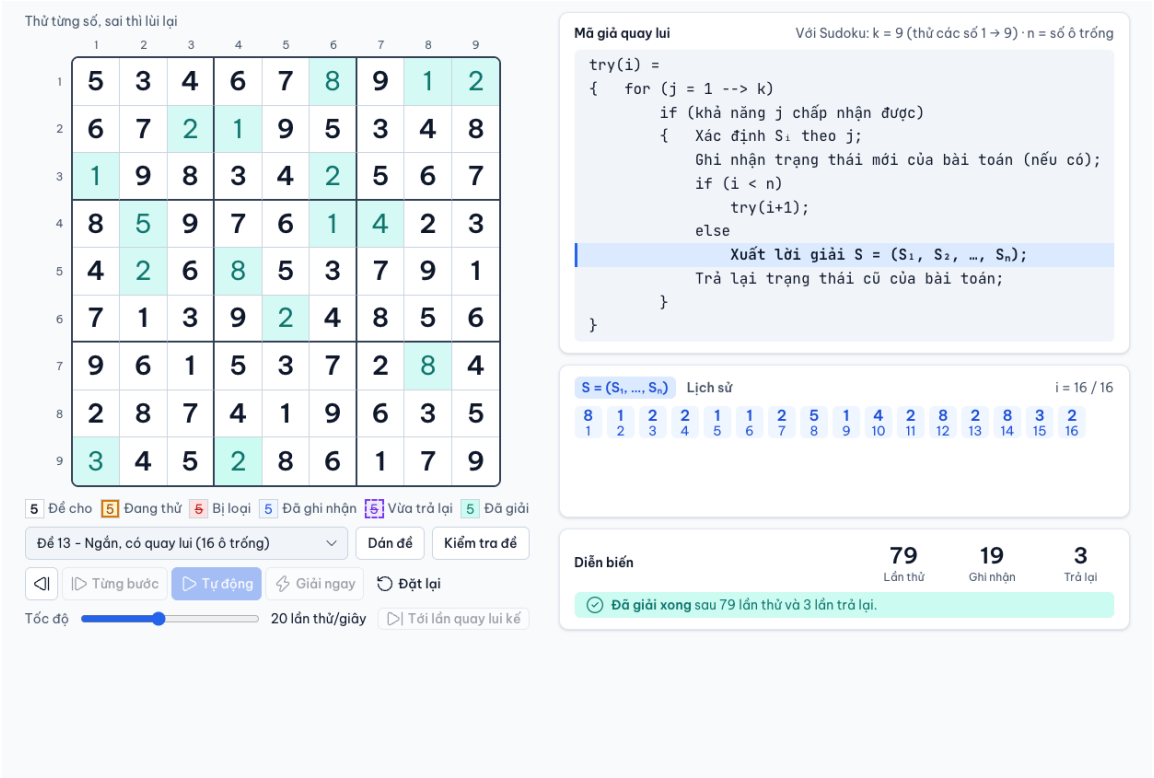
Ô $(1, 8)$ chưa hết số, nên thử tiếp từ số 2. Số 2 trùng ô $(1, 6)$, số 8 trùng khối 3, các số còn lại trùng hàng 1. Ô $(1, 8)$ hết số lần thứ hai, nên ở lần thử 29 (lần bấm 38) thuật toán lùi tiếp thêm một ô: trả lại ô $(1, 6)$, xóa số 2 (Hình 5.4). Đây là quay lui dây chuyền: ô $(1, 9)$ hết số làm ô $(1, 8)$ phải đổi số, và ô $(1, 8)$ hết số làm ô $(1, 6)$ phải đổi số.



Hình 5.4: Lần bấm 38: thuật toán vừa trả lại ô (1, 6), xóa số 2. Bộ đếm: 29 lần thử, 3 ghi nhận, 3 trả lại

Giai đoạn 3: lần thử 30 đến 79, đi thẳng tới lời giải

Ô $(1, 6)$ thử số 3 đến 7, cả năm số đều trùng hàng 1. Số 8 chấp nhận được ở lần thử 35. Từ đây thuật toán không quay lui lần nào nữa. Ô $(1, 8)$ nhận số 1, ô $(1, 9)$ nhận số 2, rồi 13 ô còn lại lần lượt nhận số. Ở lần thử 79 (lần bấm 120), ô trống cuối cùng $(9, 4)$ nhận số 2, và thuật toán xuất lời giải (Hình 5.5).



Hình 5.5: Lần bấm 120: đã giải xong. Bộ đếm: 79 lần thử, 19 ghi nhận, 3 trả lại

Kết quả cuối cùng là lời giải ở Hình 5.1(b), với $S = (8, 1, 2, 2, 1, 1, 2, 5, 1, 4, 2, 8, 2, 8, 3, 2)$. Ba lần quay lui tốn 34 lần thử, gần một nửa tổng số. Đó là cái giá của việc thử theo thứ tự cố định: thuật toán không "nhìn" trước như người giải bằng tay. Với cùng đề này, cách điền ô ít khả năng nhất trước (mục 4.5) chỉ cần 52 lần thử và không quay lui lần nào.

5.3 Bảng toàn bộ các bước

Bảng 5.2: Toàn bộ các bước giải đề số 13

Bấm	Lần thử	i	Ô	j	Thao tác	Lý do	S sau bước
1	1	1	(1, 6)	1	Chấp nhận	không trùng	1 . . . . .
4	2	2	(1, 8)	1	Bị loại	trùng hàng 1 ở ô (1, 6)	1 . . . . .
5	3	2	(1, 8)	2	Bị loại	trùng cột 8 ở ô (4, 8)	1 . . . . .
6	4	2	(1, 8)	3	Bị loại	trùng hàng 1 ở ô (1, 2)	1 . . . . .
7	5	2	(1, 8)	4	Bị loại	trùng hàng 1 ở ô (1, 3)	1 . . . . .
8	6	2	(1, 8)	5	Bị loại	trùng hàng 1 ở ô (1, 1)	1 . . . . .
9	7	2	(1, 8)	6	Bị loại	trùng hàng 1 ở ô (1, 4)	1 . . . . .
10	8	2	(1, 8)	7	Bị loại	trùng hàng 1 ở ô (1, 5)	1 . . . . .
11	9	2	(1, 8)	8	Bị loại	trùng khối 3 ở ô (2, 9)	1 . . . . .
12	10	2	(1, 8)	9	Bị loại	trùng hàng 1 ở ô (1, 7)	1 . . . . .
13	10	1	(1, 6)	1	Trả lại	ô sau hết số	. . . . .

Bấm	Lần thử	i	Ô	j	Thao tác	Lý do	S sau bước
14	11	1	(1, 6)	2	Chấp nhận	không trùng	2 . . . . .
17	12	2	(1, 8)	1	Chấp nhận	không trùng	2 1 . . . . .
20	13	3	(1, 9)	1	Bị loại	trùng hàng 1 ở ô (1, 8)	2 1 . . . . .
21	14	3	(1, 9)	2	Bị loại	trùng hàng 1 ở ô (1, 6)	2 1 . . . . .
22	15	3	(1, 9)	3	Bị loại	trùng hàng 1 ở ô (1, 2)	2 1 . . . . .
23	16	3	(1, 9)	4	Bị loại	trùng hàng 1 ở ô (1, 3)	2 1 . . . . .
24	17	3	(1, 9)	5	Bị loại	trùng hàng 1 ở ô (1, 1)	2 1 . . . . .
25	18	3	(1, 9)	6	Bị loại	trùng hàng 1 ở ô (1, 4)	2 1 . . . . .
26	19	3	(1, 9)	7	Bị loại	trùng hàng 1 ở ô (1, 5)	2 1 . . . . .
27	20	3	(1, 9)	8	Bị loại	trùng cột 9 ở ô (2, 9)	2 1 . . . . .
28	21	3	(1, 9)	9	Bị loại	trùng hàng 1 ở ô (1, 7)	2 1 . . . . .
29	21	2	(1, 8)	1	Trả lại	ô sau hết số	2 . . . . .
30	22	2	(1, 8)	2	Bị loại	trùng hàng 1 ở ô (1, 6)	2 . . . . .
31	23	2	(1, 8)	3	Bị loại	trùng hàng 1 ở ô (1, 2)	2 . . . . .
32	24	2	(1, 8)	4	Bị loại	trùng hàng 1 ở ô (1, 3)	2 . . . . .
33	25	2	(1, 8)	5	Bị loại	trùng hàng 1 ở ô (1, 1)	2 . . . . .
34	26	2	(1, 8)	6	Bị loại	trùng hàng 1 ở ô (1, 4)	2 . . . . .
35	27	2	(1, 8)	7	Bị loại	trùng hàng 1 ở ô (1, 5)	2 . . . . .
36	28	2	(1, 8)	8	Bị loại	trùng khối 3 ở ô (2, 9)	2 . . . . .
37	29	2	(1, 8)	9	Bị loại	trùng hàng 1 ở ô (1, 7)	2 . . . . .
38	29	1	(1, 6)	2	Trả lại	ô sau hết số	. . . . .
39	30	1	(1, 6)	3	Bị loại	trùng hàng 1 ở ô (1, 2)	. . . . .
40	31	1	(1, 6)	4	Bị loại	trùng hàng 1 ở ô (1, 3)	. . . . .
41	32	1	(1, 6)	5	Bị loại	trùng hàng 1 ở ô (1, 1)	. . . . .
42	33	1	(1, 6)	6	Bị loại	trùng hàng 1 ở ô (1, 4)	. . . . .
43	34	1	(1, 6)	7	Bị loại	trùng hàng 1 ở ô (1, 5)	. . . . .
44	35	1	(1, 6)	8	Chấp nhận	không trùng	8 . . . . .
47	36	2	(1, 8)	1	Chấp nhận	không trùng	8 1 . . . . .
50	37	3	(1, 9)	1	Bị loại	trùng hàng 1 ở ô (1, 8)	8 1 . . . . .
51	38	3	(1, 9)	2	Chấp nhận	không trùng	8 1 2 . . . . .
54	39	4	(2, 3)	1	Bị loại	trùng cột 3 ở ô (7, 3)	8 1 2 . . . . .
55	40	4	(2, 3)	2	Chấp nhận	không trùng	8 1 2 2 . . . . .
58	41	5	(2, 4)	1	Chấp nhận	không trùng	8 1 2 2 1 . . . . .
61	42	6	(3, 1)	1	Chấp nhận	không trùng	8 1 2 2 1 1 . . . . .
64	43	7	(3, 6)	1	Bị loại	trùng hàng 3 ở ô (3, 1)	8 1 2 2 1 1 . . . . .
65	44	7	(3, 6)	2	Chấp nhận	không trùng	8 1 2 2 1 1 2 . . . . .
68	45	8	(4, 2)	1	Bị loại	trùng cột 2 ở ô (6, 2)	8 1 2 2 1 1 2 . . . . .
69	46	8	(4, 2)	2	Bị loại	trùng hàng 4 ở ô (4, 8)	8 1 2 2 1 1 2 . . . . .
70	47	8	(4, 2)	3	Bị loại	trùng hàng 4 ở ô (4, 9)	8 1 2 2 1 1 2 . . . . .
71	48	8	(4, 2)	4	Bị loại	trùng cột 2 ở ô (9, 2)	8 1 2 2 1 1 2 . . . . .
72	49	8	(4, 2)	5	Chấp nhận	không trùng	8 1 2 2 1 1 2 5 . . . . .
75	50	9	(4, 6)	1	Chấp nhận	không trùng	8 1 2 2 1 1 2 5 1 . . . . .
78	51	10	(4, 7)	1	Bị loại	trùng hàng 4 ở ô (4, 6)	8 1 2 2 1 1 2 5 1 . . . . .
79	52	10	(4, 7)	2	Bị loại	trùng hàng 4 ở ô (4, 8)	8 1 2 2 1 1 2 5 1 . . . . .
80	53	10	(4, 7)	3	Bị loại	trùng hàng 4 ở ô (4, 9)	8 1 2 2 1 1 2 5 1 . . . . .

Bấm	Lần thử	i	Ô	j	Thao tác	Lý do	S sau bước
81	54	10	(4, 7)	4	Chấp nhận	không trùng	8 1 2 2 1 1 2 5 1 4 . . . . .
84	55	11	(5, 2)	1	Bị loại	trùng hàng 5 ở ô (5, 9)	8 1 2 2 1 1 2 5 1 4 . . . . .
85	56	11	(5, 2)	2	Chấp nhận	không trùng	8 1 2 2 1 1 2 5 1 4 2 . . . .
88	57	12	(5, 4)	1	Bị loại	trùng hàng 5 ở ô (5, 9)	8 1 2 2 1 1 2 5 1 4 2 . . . .
89	58	12	(5, 4)	2	Bị loại	trùng hàng 5 ở ô (5, 2)	8 1 2 2 1 1 2 5 1 4 2 . . . .
90	59	12	(5, 4)	3	Bị loại	trùng hàng 5 ở ô (5, 6)	8 1 2 2 1 1 2 5 1 4 2 . . . .
91	60	12	(5, 4)	4	Bị loại	trùng hàng 5 ở ô (5, 1)	8 1 2 2 1 1 2 5 1 4 2 . . . .
92	61	12	(5, 4)	5	Bị loại	trùng hàng 5 ở ô (5, 5)	8 1 2 2 1 1 2 5 1 4 2 . . . .
93	62	12	(5, 4)	6	Bị loại	trùng hàng 5 ở ô (5, 3)	8 1 2 2 1 1 2 5 1 4 2 . . . .
94	63	12	(5, 4)	7	Bị loại	trùng hàng 5 ở ô (5, 7)	8 1 2 2 1 1 2 5 1 4 2 . . . .
95	64	12	(5, 4)	8	Chấp nhận	không trùng	8 1 2 2 1 1 2 5 1 4 2 8 . . .
98	65	13	(6, 5)	1	Bị loại	trùng hàng 6 ở ô (6, 2)	8 1 2 2 1 1 2 5 1 4 2 8 . . .
99	66	13	(6, 5)	2	Chấp nhận	không trùng	8 1 2 2 1 1 2 5 1 4 2 8 2 . .
102	67	14	(7, 8)	1	Bị loại	trùng hàng 7 ở ô (7, 3)	8 1 2 2 1 1 2 5 1 4 2 8 2 . .
103	68	14	(7, 8)	2	Bị loại	trùng hàng 7 ở ô (7, 7)	8 1 2 2 1 1 2 5 1 4 2 8 2 . .
104	69	14	(7, 8)	3	Bị loại	trùng hàng 7 ở ô (7, 5)	8 1 2 2 1 1 2 5 1 4 2 8 2 . .
105	70	14	(7, 8)	4	Bị loại	trùng hàng 7 ở ô (7, 9)	8 1 2 2 1 1 2 5 1 4 2 8 2 . .
106	71	14	(7, 8)	5	Bị loại	trùng hàng 7 ở ô (7, 4)	8 1 2 2 1 1 2 5 1 4 2 8 2 . .
107	72	14	(7, 8)	6	Bị loại	trùng hàng 7 ở ô (7, 2)	8 1 2 2 1 1 2 5 1 4 2 8 2 . .
108	73	14	(7, 8)	7	Bị loại	trùng hàng 7 ở ô (7, 6)	8 1 2 2 1 1 2 5 1 4 2 8 2 . .
109	74	14	(7, 8)	8	Chấp nhận	không trùng	8 1 2 2 1 1 2 5 1 4 2 8 2 8 . .
112	75	15	(9, 1)	1	Bị loại	trùng hàng 9 ở ô (9, 7)	8 1 2 2 1 1 2 5 1 4 2 8 2 8 . .
113	76	15	(9, 1)	2	Bị loại	trùng cột 1 ở ô (8, 1)	8 1 2 2 1 1 2 5 1 4 2 8 2 8 . .
114	77	15	(9, 1)	3	Chấp nhận	không trùng	8 1 2 2 1 1 2 5 1 4 2 8 2 8 3 .
117	78	16	(9, 4)	1	Bị loại	trùng hàng 9 ở ô (9, 7)	8 1 2 2 1 1 2 5 1 4 2 8 2 8 3 .
118	79	16	(9, 4)	2	Chấp nhận	không trùng	8 1 2 2 1 1 2 5 1 4 2 8 2 8 3 2
120	79				Có lời giải	đủ 16 ô	8 1 2 2 1 1 2 5 1 4 2 8 2 8 3 2

Chương 6

Quá trình thiết kế và xây dựng

Em dùng Claude Code để lập kế hoạch, viết code, viết test và đo đạc. Phần chọn hướng, duyệt kết quả và quyết định cuối cùng do em giữ. Chương này kể lại từng giai đoạn theo thứ tự thời gian, từ ngày 3/10 đến ngày 12/10/2026.

Nguồn của chương là nhật ký làm việc `work-log.md`, em ghi ngay sau mỗi buổi, và transcript của Claude Code lưu trên máy. Mọi prompt trong chương chép nguyên văn từ transcript, kể cả lỗi chính tả. Mỗi giai đoạn ghi công cụ, prompt, kết quả, vấn đề gặp phải và cách em kiểm chứng.

6.1 Công cụ và cách làm việc

Bảng 6.1 liệt kê các công cụ, thư viện và model em dùng.

Bảng 6.1: Công cụ, thư viện và model

Công cụ	**Vai trò**
Claude Code (tiện ích VS Code, phiên bản 2.1.287 đến 2.1.295), model Claude Opus 5.5	Lập kế hoạch, viết code website và các script, viết test, đo đạc, mở website trên Chrome để chụp màn hình và tự soát, viết nháp chữ trên website và report
Claude Design (trên claude.ai)	Vẽ 19 khung hình giao diện từ một prompt do Claude Code soạn
Một phiên Claude Code riêng	Đóng vai người dùng phổ thông, nhận xét website
React 19, React Router 8, TypeScript 6, Vite 8	Website chạy hoàn toàn trên trình duyệt, không có máy chủ
Vitest 5, Playwright 1.63, oxlint	Test đơn vị, test trên Chrome thật, soát lỗi code
Python 3.9	Các bản cài đặt tham chiếu để đối chiếu, script sinh bộ testcase
Yarn 4.18.1, lucide-react, Fontsource	Quản lý gói, biểu tượng, phông Be Vietnam Pro và JetBrains Mono
XeLaTeX	Report này

Mỗi việc đi qua cùng một vòng:

- Em nêu yêu cầu bằng một prompt ngắn.
- Claude đọc code hiện có, đo đạc khi cần, rồi đề xuất phương án. Chỗ nào có nhiều lựa chọn thì Claude hỏi lại em.
- Em chọn. Claude làm, chạy test, mở website trên Chrome thật để chụp màn hình và tự soát.
- Em xem kết quả, góp ý tiếp, rồi ghi vào nhật ký.

Ngay prompt lập kế hoạch đầu tiên, em đặt luật văn phong cho Claude. Em giữ nguyên bảy dòng tiếng Anh:

Prompt · 3/10, kèm theo prompt lập kế hoạch

- Cut hype, buzzwords, empty phrasing and anything that reads like marketing copy.
- Write with a clear position. Fewer “it depends” hedges, fewer generic recommendations.
- Mix short, medium and long sentences. Uniform, too-perfect rhythm reads as machine-written.
- Replace vague claims with examples, scenarios and real detail — numbers, file paths, commit hashes.

- Use plain, natural words, as if explaining to someone without much background in the topic.
- Drop generic openers, filler, repeated transitions and summary paragraphs that add nothing.
- Match the user's own voice: sample a few things they wrote, then follow that word choice, sentence rhythm and personality.

Luật về code em đặt ngay sau đó, và giữ suốt dự án:

Prompt · 3/10

Remember everything in code should be in English, code must be easy to read, easy to understand, must be explicit, can explain itself without comment. Do you think we can use React for the website?

6.2 Chọn đề và lập kế hoạch (3/10)

Em phân vân giữa Dijkstra và Sudoku.

Prompt · 3/10

Bạn nghĩ sao về Dijkstra, tôi cũng thích Sudoku

Claude khuyên bỏ Dijkstra vì trên lớp chưa học, cô chỉ nhắc tới như bài của khóa trước. Em chọn Sudoku. Lý do: em đã tự cài đặt quay lui theo mô hình 1 cho bài này từ trước, và bài toán có đủ ba động tác thử, loại, quay lui để nhìn thấy trên hình.

Em yêu cầu lập kế hoạch đầy đủ trước khi viết dòng code nào:

Prompt · 3/10

Ok, tôi đồng ý với Sudoku, nhưng chúng ta sẽ không thực hiện liền mà cần lên kế hoạch phân tích toàn bộ bài toán trước khi viết những dòng code đầu tiên. Vì vậy, hãy tạo 1 file markdown chứa toàn bộ chi tiết kế hoạch cho bài tập lớn này, lưu ý, giọng văn của bạn phải tuân thủ tuyệt đối theo yêu cầu của tôi dưới đây [bảy dòng luật văn phong ở mục trên]

Claude viết `plan.md` gồm 12 mục. Trước khi viết, Claude chạy bản cài đặt Python của em trên 7 đề để lấy số đo thật. Các quyết định thiết kế dựa vào số đo đó. Đề Arto Inkala cần 445.778 lần thử, nên không thể chỉ có hoạt ảnh chậm: website phải có thêm nút giải một mạch.

Prompt · 3/10

5.1. I wanna build a completely website and deploy to Netlify or any supported free server
6. UI must be nice, modern, can I use Claude Design for this?

Prompt · 3/10

Yes, file name should be in English as well. Don't need to care about zip and offline for now. Just let me know if we use React, does it affect to the performance?

Câu hỏi hiệu năng có câu trả lời bằng số đo. Trên đề Inkala, bản đệ quy thường chạy hết 3,7 ms. Bản viết thành generator, để giao diện dừng được ở từng bước, mất 623 ms. Phần chậm nằm ở generator, không nằm ở React. Từ số đo này, kế hoạch chốt cách vẽ: mỗi khung hình lấy một loạt bước rồi mới vẽ lại một lần, thay vì vẽ lại sau từng bước.

6.3 Thiết kế giao diện bằng Claude Design (3/10 - 4/10)

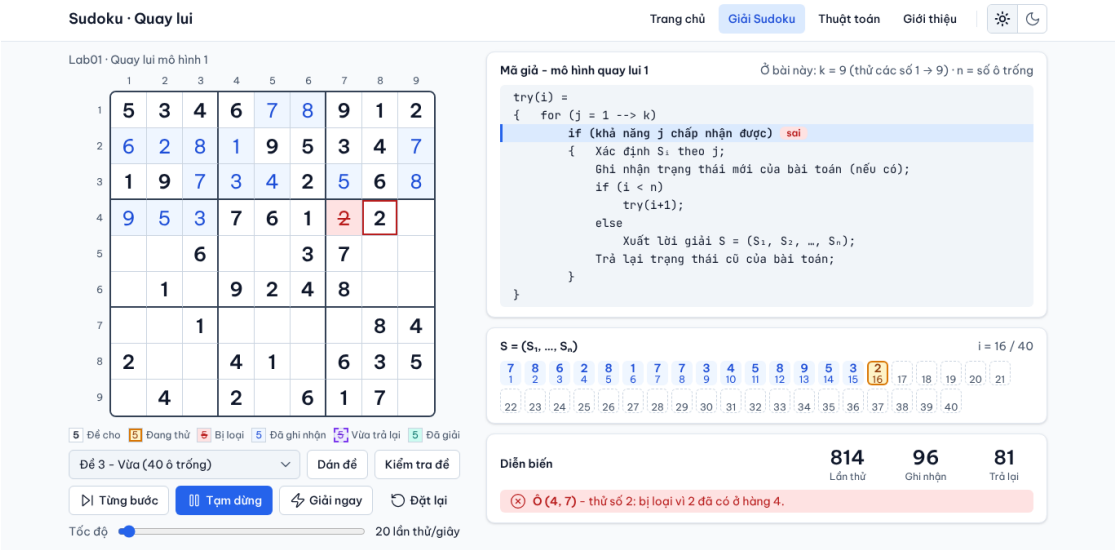
Prompt · 3/10

Can we build a very detail prompt then I can use to talk with Claude Design for the mockup? We should have the color pallet for the website, it should be mordern, no violate design rule.

Claude Code soạn file `design/claude-design-prompt.md`. File này có bốn điểm em thấy quan trọng:

- **Bảng màu có đo độ tương phản** theo chuẩn WCAG AA. Một màu xám chỉ đạt 4,34:1, thiếu so với mức 4,5:1, nên Claude bỏ màu đó.
- **Mỗi trạng thái ô có dấu hiệu ngoài màu**, ví dụ số không hợp lệ có thêm nét gạch ngang, để người khó phân biệt đỏ và xanh vẫn đọc được.
- **Trạng thái lưới lấy từ lần chạy thật** của đề "Vừa" tại lần thử 803, lần thử 814 và lúc giải xong, không vẽ phỏng đoán.
- **Mã giả chép đúng từng chữ slide** mô hình 1 của cô.

Em đưa prompt vào Claude Design và trong lúc làm yêu cầu thêm nút chuyển Sáng và Tối. Claude Design trả về 19 khung hình, file màu `tokens.css` và hướng dẫn bàn giao. Hình 6.1 là một khung trong số đó.



Hình 6.1: Khung 04 của bản thiết kế: lần thử 814 của đề "Vừa", số 2 bị loại ở ô (4, 7)

Prompt · 4/10

Claude Design has finished their job and hand-off design in the folder design, you can take a look at this and let me know what your plan next?

Trước khi code theo bản thiết kế, Claude chạy lại thuật toán tới đúng thời điểm của từng khung để soát số. Khung 04 (lần thử 814), khung 05 (lần thử 803) và khung 17 (đề Inkala ở lần thử 200.000) đều khớp từng con số.

6.4 Chuyển thuật toán sang TypeScript (4/10)

Claude viết lại bản cài đặt của em bằng TypeScript, đặt trong thư mục `website/src/solver`. Hàm `tryStep(i)` giữ đúng các bước của mô hình 1. Khác biệt duy nhất: ở mỗi bước, hàm phát ra một sự kiện để giao diện vẽ. Bảng đối chiếu từng dòng nằm ở Chương 4.

Đáp án cho test là bản Python thêm ba bộ đếm: số lần thử, số lần ghi nhận, số lần trả lại. Hai bản phải cho cùng lời giải và cùng ba con số trên mọi đề. Lần đầu có 29 test, tất cả đạt.

Tất cả đạt ngay lần đầu chưa chứng minh được gì, nên Claude thử làm sai thuật toán xem test có bắt được không:

- **Bỏ số 9** khỏi vòng thử: 10 test báo lỗi.
- **Kiểm tra cột trước hàng**: không test nào báo lỗi. Lý do: ba thời điểm chụp dùng làm đáp án đều chỉ trùng ở hàng. Claude thêm một test duyệt toàn bộ 1.682 lần bị loại của đề "Vừa", so cả lý do bị loại. Từ đó test bắt được lỗi này.

Giai đoạn này có hai sự cố nhỏ với trình quản lý gói:

Prompt · 4/10

Can we use yarn v2 instead of npm?

- Yarn 2 đã ngừng hỗ trợ, nên Claude dùng Yarn 4.18.1, cùng kiến trúc với bản 2. Lần chạy đầu, Yarn ghi nhầm ba file vào thư mục home vì ở đó có một file `yarn.lock` rỗng từ ngày 26/8. Claude xóa đúng ba thứ vừa tạo, giữ nguyên file cũ, rồi chạy lại trong thư mục dự án.
- Yarn 4 từ chối cài lucide-react 1.51.0 vì bản này phát hành chưa đủ 24 giờ. Claude giữ lớp bảo vệ đó và dùng bản 1.50.0.

6.5 Xây dựng trang Giải Sudoku (4/10)

Website lúc đầu còn ghi chữ "Wecode" trên giao diện. Em yêu cầu bỏ:

Prompt · 4/10

Yes, please go head. But before moving forward, I don't think label "Wecode" is ok, we can use another label, right?

Prompt · 4/10

I mean don't want any text relates to "Lab" "Wecode" like we're studying. Treat this website like the one to be used for people understand the algorithm, not my exercises

Từ đây website viết cho người muốn hiểu thuật toán. Mã giả vẫn chép đúng slide, chỉ tiêu đề đổi thành chữ chung như "Mã giả quay lui".

Trang Giải Sudoku có lưới gõ số được, chú thích màu, chọn đề mẫu, dán đề, các nút Từng bước, Tự động, Giải ngay, Đặt lại, thanh tốc độ, mã giả tô sáng dòng đang chạy, bảng $S = (S_1, \ldots, S_n)$, bộ đếm và câu thông báo.

Bảng lịch sử

Prompt · 4/10

I'm thinking about "Diễn biến" table should has history or last 10~15 steps, and can we trace back through history? Does it cost more performance?

Claude không đoán con số mà đo. Trên ba đề "Vừa", Wikipedia và Inkala, một nhánh sai dài trung vị 21 đến 31 bước. Giữ 100 bước thì thấy trọn khoảng 80 phần trăm số nhánh sai. Giữ 15 bước chỉ thấy khoảng 30 phần trăm. Em chọn 100.

Cách làm cũng tránh tốn bộ nhớ. Website không lưu bản sao lưới ở mỗi bước, chỉ lưu sự kiện. Khi xem lại, website "hoàn tác" ngược từ trạng thái hiện tại. Test kiểm tra lùi từ 1 đến 300 sự kiện đều ra đúng trạng thái thật lúc đó.

Bố cục, chữ giao diện và test trên trình duyệt

Prompt · 4/10

Screenshot: The solver screen, the components are very tight while we still have a lot of blank space below, can we make it fit the screen (responsive height)?

Prompt · 4/10

I don't like put css in the same folder with tsx file. Any suggestion for folder structure?
And for the label, we should store them in a json file to preparing for localization/i18n in the future

Lưới giờ co giãn theo chiều cao màn hình, từ ô 44 px tới 96 px. CSS tách sang thư mục riêng. Khoảng 300 dòng chữ giao diện chuyển vào một file JSON. Claude kiểm tra không còn chữ tiếng Việt nào trong code ngoài file đó.

Claude viết một script điều khiển Chrome để bấm tới đúng lần thử của từng khung thiết kế rồi chụp màn hình so sánh. Cách soát này tìm ra các lỗi sau, tất cả đã sửa:

- Thanh trên cùng cao 57 px thay vì 56 px, làm trang dư 1 px.
- Đề sai thì bảng S biến mất.
- Đóng hộp dán đề thì con trỏ bàn phím không quay về nút "Dán đề".
- 81 ô lưới vẽ lại ở mọi khung hình, vì hàm truyền xuống ô không cố định.
- Mục lục trang Thuật toán tô sai mục khi cuộn.
- Mở tab Lịch sử làm trang cao tới 2.998 px.

Sau đó Claude chuyển các bước soát này thành 31 test Playwright chạy trên Chrome thật. Để chắc bộ test có tác dụng, Claude đưa lại lỗi 57 px của ngày đầu. Ba test báo đúng "Expected 680, Received 681".

6.6 Góp ý từ một phiên Claude đóng vai người dùng (4/10)

Em chưa có người thật dùng thử. Em mở một phiên Claude Code riêng và nhờ nó đóng vai người dùng phổ thông:

Prompt · 4/10, phiên riêng

As an end user (normal end user), how you rate my sudoku website? And feedback from your side for UI, for functional, any things else?

Phiên đó chỉ trả lời, không sửa file nào. Em dán nguyên bản nhận xét sang phiên chính:

Prompt · 4/10

A feedback from an end-user [nguyên văn bản nhận xét]
We can skip the video demo/source code for now. You can address the feedback from the end-user

Claude đo lại trước khi sửa. Bảng S ở màn hình 1366 × 768 cắt đôi hàng ô 33 đến 40, chú thích màu cao 12 px. Hai điểm này đúng như nhận xét. Các thay đổi sau đó:

- Bảng S chỉ hiện trọn hàng, có thanh cuộn luôn thấy được.
- Rê chuột lên ô nào thì hiện nghĩa màu của ô đó. Chú thích màu lớn theo lưới.
- Thêm nút lùi một bước cạnh nút Từng bước, và nút tới lần quay lui kế tiếp.
- Chạy tự động thì dừng ở lần quay lui đầu tiên và hỏi người xem đoán bước tiếp theo.
- Trang chủ có lưới chạy lại thật các bước 783 đến 814 của đề "Vừa", đứng yên nếu máy bật giảm chuyển động.

Một góp ý em không làm theo. Nhận xét nói đề Wikipedia chạy tự động mất nửa giờ. Con số đó đúng ở tốc độ mặc định 20 lần thử mỗi giây. Ở tốc độ tối đa 1.000 lần thử mỗi giây, đề này chạy hết 38 giây. Em giữ tốc độ tối đa, thêm nút tới lần quay lui kế để người xem tới khoảnh khắc đáng xem nhanh hơn.

Em cũng tự sửa vài chỗ chữ, ví dụ:

Prompt · 4/10

The word "Người làm" is very weird, why you don't use "Tác giả" or more meaningful word?

6.7 Yêu cầu chính thức và đổi góc nhìn (6/10)

Prompt · 6/10

Cô đã cập nhật yêu cầu cho final-project rồi, review qua xem chúng ta còn cần gì. Yêu cầu này sẽ overwrite yêu cầu ở buổi 8 [nguyên văn yêu cầu của cô]

Claude đối chiếu yêu cầu mới với kế hoạch và code. Phần demo đã đủ. Phần thiếu gồm bộ testcase, report khoảng 10 mục, video phải đi qua mọi chức năng, và phần sáng tạo: kế hoạch cũ ghi "không dùng thuật toán khác", trong khi tiêu chí mới chấm điểm việc giải nhiều cách rồi so sánh. Claude cũng tìm ra một lỗi trong kế hoạch: dòng "Dừng khi $k - i = 0$" phải là $n - i = 0$.

Prompt · 6/10

1. Đồng ý với gợi ý của bạn
2. Report bằng LaTeX
3. Sẽ quyết định sau khi chúng ta hoàn tất mọi thứ

Prompt · 6/10

Không nhất thiết cứ phải mention Lab 01 hay Wecode, hãy mở rộng phạm vi ra. Treat mọi thứ như chúng ta đang dựng 1 website để dạy học và chia sẻ kiến thức vậy

Claude hỏi lại "mở rộng" theo hướng nào: chỉ đổi góc nhìn, thêm trang bài học quay lui, hay thêm bài toán khác. Em chọn đổi góc nhìn và vẫn giữ Sudoku, vì 12 ngày không đủ cho bài toán thứ hai. Em chọn thêm cách "điền ô ít khả năng nhất trước" (MRV) để so sánh với mô hình 1.

6.8 Bộ testcase (6/10)

Claude viết script sinh 294 đề chia 6 nhóm, đáp án lấy từ hai chương trình Python độc lập với website. Chương 7 trình bày chi tiết. Ở đây em kể các vấn đề gặp phải.

Vòng lặp không dừng. Lần chạy đầu treo hơn 10 phút. Claude đo từng nhóm và tìm ra vòng sinh đề "đúng 2 lời giải" lặp mãi. Lý do: mọi lưới "ngẫu nhiên" đều sinh từ một lưới mẫu rồi hoán vị hàng, cột và đổi tên số. Cả họ lưới đó không bao giờ có bốn ô hình chữ nhật đổi chỗ được. Lỗi này còn lộ ra một điểm yếu: bộ đề kém đa dạng hơn em tưởng. Claude đổi sang sinh lưới đầy bằng quay lui với thứ tự số xáo trộn.

Bộ test lỏng. Mọi đề đạt ngay lần đầu. Claude kiểm lại bằng bốn lỗi cố ý, và lần đầu test chỉ bắt được hai lỗi:

- "Kiểm tra cột trước hàng" lọt qua, vì đáp án chỉ ghi lời giải và ba bộ đếm. Claude thêm ba cột: số lần bị loại vì hàng, vì cột, vì khối.
- "Bỏ qua khối khi kiểm tra đề" làm bộ giải chạy mãi trên đề trùng khối. Claude đặt giới hạn số lần thử cho mỗi đề, bằng đúng số trong đáp án.

Sau khi vá, test bắt được cả bốn lỗi, lần lượt 229, 5, 1 và 262 đề báo sai. Trong lúc thử, một lệnh chạy nền vẫn tiếp tục sửa code sai sau khi Claude đã dừng nó. Claude dừng hẳn tiến trình đó và khôi phục code từ git trước khi làm tiếp.

Prompt · 6/10

Wow, there are too much files .in and .out

294 đề nhân hai file là 588 file. Em chọn gom mỗi nhóm thành một file văn bản, mỗi đề một khối gồm mô tả, đề và kết quả mong đợi.

Prompt · 6/10

Why it takes so long?

Hai đề chậm nhất chạy hết 18 phút 45 giây trên bộ giải của website: đề chống quay lui cần 622.577.597 lần thử, đề 17 số cho sẵn cần 239.312.393 lần. Phần chậm nằm ở generator. Em giữ

nguyên, vì bộ test phải chạy đúng bộ giải người xem thấy trên website. Hai đề này không chạy mặc định.

6.9 Trang So sánh (6/10)

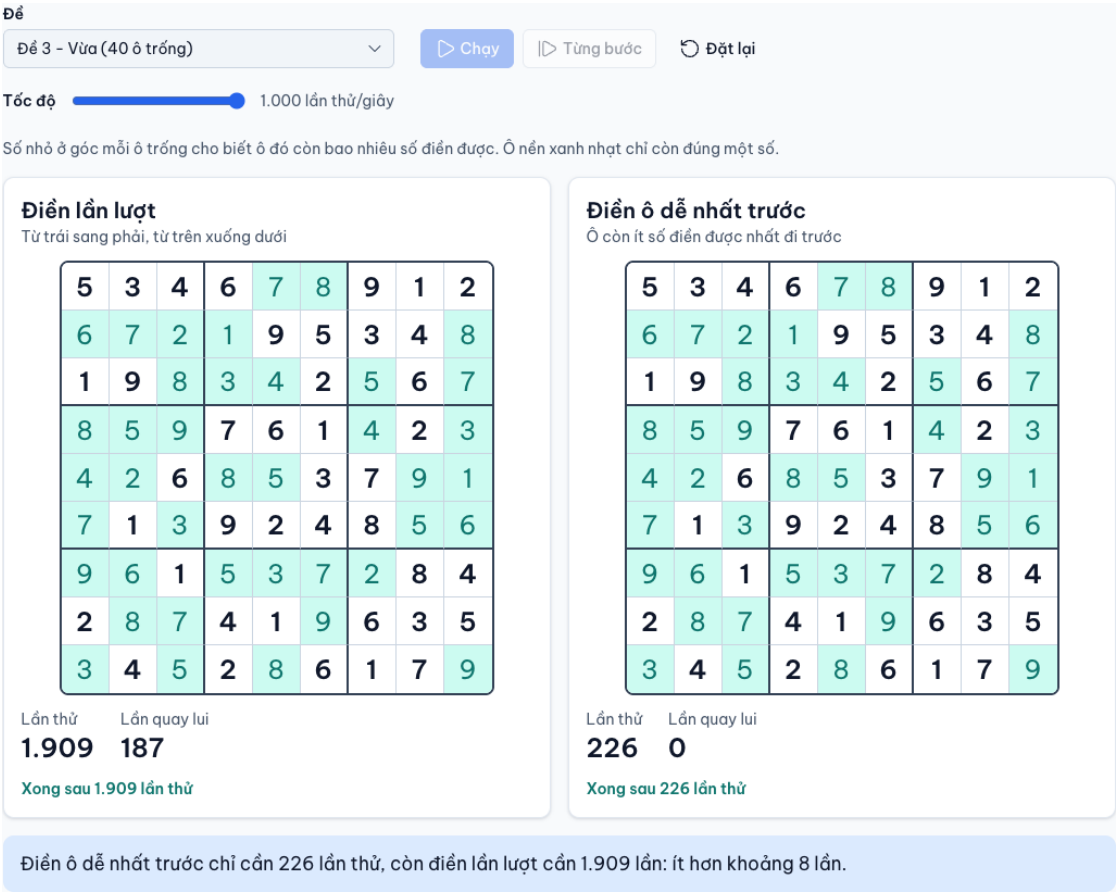
Claude viết phần thuật toán trước, đo xong mới làm giao diện. Ba cách kiểm tra một số có hợp lệ không (đọc lại lưới, ba mảng đánh dấu, bitmask) cho đúng cùng số lần thử trên cả 12 đề. Cách điền ô ít khả năng nhất trước giảm mạnh số lần thử: đề chống quay lui giảm từ 622.577.597 xuống 407.156. Trên đề Inkala, cách này lại chậm hơn bản thường: 21,5 ms so với 5,2 ms, vì mỗi bước phải đếm lại khả năng của mọi ô trống.

Phiên bản đầu của trang có bảng số liệu và hai biểu đồ thanh. Em mở thử và không hiểu:

Prompt · 6/10, kèm ảnh chụp màn hình

Do we have any visualize for Comparing page? As a normal user, I don't understand any word of this page?

Hai biểu đồ để trống cho tới khi bấm Đo, và chữ toàn thuật ngữ. Claude đề xuất ba kiểu minh họa. Em chọn cuộc đua giữa hai lưới (Hình 6.2): cùng đề, cùng tốc độ, lưới trái điền lần lượt, lưới phải điền ô dễ nhất trước, mỗi ô trống hiện số khả năng còn lại. Thuật ngữ đổi sang lời thường: "Nhìn lại lưới", "Sổ ghi chép", "Dãy công tắc".



Hình 6.2: Cuộc đua trên đề "Vừa": điền ô dễ nhất trước xong sau 226 lần thử, điền lần lượt cần 1.909 lần

Để có đáp án độc lập, Claude viết thêm một bản Python cho cách điền ô ít khả năng nhất trước. Bản này khớp bản TypeScript trên cả 12 đề.

6.10 Trang Mọi lời giải (6/10)

Mô hình 1 trên slide vốn không dừng ở lời giải đầu: xuất xong một lời giải, nó trả lại trạng thái cũ rồi thử tiếp. Em chọn đặt chế độ "tìm hết" ở một trang riêng để trang Giải Sudoku giữ nguyên.

Khi chạy bản Python đếm lời giải, Claude phát hiện đề mẫu "Vừa", đề mặc định của trình giải, có 4 lời giải chứ không phải 1. Bộ đếm độc lập trong bộ testcase cũng ra 4. Em sửa lại các chỗ tài liệu ghi sai điều này.

Claude kiểm chéo thêm: chế độ tìm hết đếm lại số lời giải của 251 đề trong bộ testcase. 239 đề tìm hết trong giới hạn 300 nghìn lần thử, và cả 239 đề khớp bộ đếm độc lập. Lỗi tìm ra khi chụp màn hình: ở màn hình rộng 820 px, chữ trong menu sáu mục dồn xuống hai dòng. Claude đo thấy sáu mục vừa một dòng từ 880 px, nên cho menu thu gọn dưới 900 px.

6.11 Quy tắc văn phong và viết lại chữ trên website (6/10)

Em dán ba bộ quy tắc văn phong, bắt đầu bằng:

Prompt · 6/10

Nguyên tắc dành cho bạn khi làm việc với câu chữ, hãy lưu vào system
• Quy tắc chống văn phong AI (Anti-AI Patterns): Cấm dùng các từ ngữ sáo rỗng, cầu kỳ quá mức (ví dụ: "trong bức tranh toàn cảnh", "tóm lại là", "không thể phủ nhận rằng"), ưu tiên câu ngắn, nhịp điệu linh hoạt.
• Quy chuẩn mạch lạc (Flow): Hướng dẫn cách chuyển ý tự nhiên giữa các đoạn, đi từ ví dụ thực tế rồi mới rút ra kết luận thay vì áp đặt lý thuyết khô khan

Claude chỉ ra ba chỗ các bộ quy tắc mâu thuẫn nhau hoặc mâu thuẫn với bài làm. Cấu trúc "câu chuyện trước" ngược với "kết luận trước". Luật "không đưa code" đụng tới mã giả cô chấm. Giọng hài hước không hợp với report. Claude đề xuất chia quy tắc theo nơi dùng: website dùng đủ cả bộ, report chỉ dùng luật về câu và bộ lọc từ.

Prompt · 6/10

Yeah, về mã giả thì bất khả xâm phạm, tôi đồng ý

Prompt · 6/10

Sao bạn không lưu vào như 1 SKILL?

Bộ quy tắc thành một skill của Claude Code, có bước tự kiểm trước khi giao chữ. Em đưa thêm một bài viết mẫu. Claude chỉ ra hai lỗi thuật toán trong bài: "điền bừa một số" (thuật toán thử lần lượt từ 1 đến 9, không có gì bừa) và "thử hết tất cả các trường hợp" (thuật toán bỏ ngay số trùng, nên đề Inkala chỉ cần 445.778 lần thử thay vì 9^{60}).

Claude viết lại khoảng 50 câu trên website. Bảng 6.2 có ba ví dụ.

Bảng 6.2: Ba câu trước và sau khi viết lại (tiêu đề trang chủ là bản sau lần sửa ngày 12/10)

Trước	**Sau**
Xem thuật toán quay lui giải Sudoku, từng bước một.	Thuật toán quay lui giải Sudoku như thế nào?
Ô (4, 7) - thử số 2: bị loại vì 2 đã có ở hàng 4.	Ô (4, 7) - thử số 2: hàng 4 đã có số 2, bỏ.
Nói cách khác, O(9^n) ở phần trên không chỉ là chuyện lý thuyết.	O(9^n) ở phần trên không chỉ nằm trên giấy: đề này bắt thuật toán đi gần hết cái cây đó.

Tiêu đề trang chủ phải sửa thêm một lần. Bản Claude viết lúc đầu là "Máy giải Sudoku cũng đoán sai. Xem nó lùi lại và sửa." Đọc lại, em thấy câu này không giống người thật viết:

Prompt · 12/10

"Máy giải Sudoku cũng đoán sai. Xem nó lùi lại và sửa." Cái câu này đọc không giống người thường một chút nào cả, nghe là biết AI rồi. Bạn thử hỏi mấy con agent khác xem, nghe thấy gớm

Claude giao cho ba agent riêng, mỗi agent đóng một vai người đọc: người viết nội dung cho trang dạy code, sinh viên năm hai, biên tập viên tiếng Việt. Ba agent chấm ba tiêu đề: bản của Claude, bản em tự viết lại ("Cùng xem thuật toán quay lui giải Sudoku như thế nào nhé!") và bản gốc. Cả ba chấm bản của em thấp nhất, 3 đến 4 trên 10, vì "Cùng…nhé!" là giọng người dẫn chương trình. Bản của Claude được 6 đến 7, nhưng cả ba cùng chỉ ra hai lỗi: chữ "nó" dịch từ "it" của tiếng Anh, và nhịp hai câu cụt kiểu khẩu hiệu. Biên tập viên còn bắt thêm lỗi trong đoạn dẫn: "mỗi cửa thử từng chìa" sai chủ ngữ, và hình ảnh dãy cửa chưa giải thích vì sao phải quay lại. Em chọn tiêu đề "Thuật toán quay lui giải Sudoku như thế nào?" và dùng nguyên đoạn dẫn biên tập viên sửa. Các lỗi này thành luật mới trong bộ quy tắc văn phong.

Lần viết lại này để lọt một lỗi. Ba câu mới có chỗ chờ điền số, nhưng code chưa truyền số vào, nên trang hiện nguyên dấu ngoặc nhọn. Không test nào bắt được. Claude tự phát hiện ngay sau khi commit, sửa, rồi thêm một test mở từng trang và khẳng định không còn chỗ chờ điền nào hiện ra. Lần đầu chính test này vẫn đạt trên bản lỗi, vì nó đọc chữ khi trang còn đang tải. Claude đổi sang chờ trang tải xong, và test báo lỗi đúng ở trang Thuật toán.

6.12 Đề minh họa và sự cố máy chậm (8/10 - 10/10)

Chương 5 cần một đề đủ ngắn để in trọn từng bước, mà vẫn có quay lui. Claude viết script tìm đề theo tiêu chí: 10 đến 16 ô trống, đúng một lời giải, 50 đến 90 lần thử, 1 đến 3 lần quay lui, có đủ ba kiểu bị loại. Script ra 12 đề. Em chọn đề 79 lần thử, 3 lần quay lui, vì cả ba lần quay lui nằm gọn trong 35 lần thử đầu trên hàng 1, và có một lần lùi dây chuyền qua hai ô. Đề này thành đề mẫu số 13 trên website. Một script khác xuất cả 83 bước bằng chính bộ giải của website, nên bảng ở Chương 5 khớp với demo.

Ngày 9/10, `yarn test` từ 9 giây thành 16 phút, mỗi lần chạy hỏng một test khác nhau vì quá giờ. Claude đo bằng một vòng lặp: cần 1,9 giây CPU mà mất 27 giây, tức máy chỉ cấp khoảng 5 đến 7 phần trăm sức chạy. Máy không bật tiết kiệm pin, cũng không có cảnh báo nhiệt. Claude nới giới hạn thời gian mỗi test lên 30 giây. Ngày 10/10 máy chạy bình thường trở lại, và toàn bộ test đạt: 723 test đơn vị, 292 testcase, 60 test trên Chrome.

6.13 Phần việc của em và phần việc của AI

Bảng 6.3 tách phần em làm và phần Claude làm.

Bảng 6.3: Phân chia công việc

Việc	**Em**	**Claude**
Chọn đề, phạm vi, công nghệ	Quyết định mọi lựa chọn	Đề xuất phương án, đo đạc để em chọn
Thuật toán	Tự cài đặt quay lui theo mô hình 1 bằng Python từ trước	Chuyển sang TypeScript, giữ đúng cấu trúc; viết thêm biến thể MRV và tìm hết
Giao diện	Duyệt bản thiết kế, thêm yêu cầu Sáng và Tối, góp ý từng màn hình	Claude Code soạn prompt; Claude Design vẽ; Claude Code code theo bản vẽ
Code website, script, test	Duyệt kết quả qua ảnh chụp và chạy thử	Viết toàn bộ code, test, script đo đạc
Chữ trên website	Đặt quy tắc văn phong, sửa các câu chưa tự nhiên	Viết nháp theo quy tắc
Kiểm chứng	Yêu cầu soát lại khi thấy lạ (trang So sánh, thời gian chạy)	Đề xuất và chạy các cách soát: bản Python độc lập, cố tình làm sai, chụp màn hình
Report	Đọc, sửa và chịu trách nhiệm nội dung	Viết bản nháp từ nhật ký và transcript

Các cách kiểm chứng em dựa vào xuyên suốt dự án:

- **Đáp án độc lập.** Mọi con số trên website và trong report đều so với bản Python viết riêng: bản tham chiếu mô hình 1, bản điền ô ít khả năng nhất trước, bản tìm hết, và bộ đếm lời giải dùng cách khác hẳn.
- **Cố tình làm sai.** Mỗi khi test đạt hết ngay lần đầu, Claude đưa lỗi vào thuật toán để xem test có bắt được không. Hai lần cách này lộ ra lỗ hổng thật.
- **Chụp màn hình trên Chrome thật** ở nhiều cỡ màn hình, sáng và tối, trên điện thoại.
- **Số đo thật thay cho phỏng đoán**, từ số bước giữ trong lịch sử tới chiều rộng menu.

Chương 7

Thiết kế thực nghiệm

Bộ testcase có 294 đề, chia 6 nhóm. Số ô trống trải từ 0 đến 81, số lần thử từ 0 đến 622.577.597. Đáp án của mọi đề lấy từ hai chương trình Python viết riêng, không lấy từ chính bộ giải đang được kiểm thử. Bảng 7.1 tóm tắt các nhóm.

Bảng 7.1: Các nhóm testcase và kết quả lần chạy ngày 10/10/2026

Nhóm	Số đề	Ô trống	Lần thử	Đạt	Thời gian lớn nhất
Nhỏ	40	1 - 5	1 - 31	40/40	0,800 ms
Thông thường	150	20 - 50	91 - 43.531	150/150	28,8 ms
Lớn, khó	45	51 - 70	587 - 622.577.597	43/43	3.069 ms
Biên	13	0 - 81	0 - 3.195	13/13	3,20 ms
Đặc biệt	21	4 - 72	9 - 37.652	21/21	31,5 ms
Không hợp lệ	25			25/25	0,100 ms

Nhóm Lớn, khó có 2 đề chậm không chạy mặc định (mục 7.2), nên cột Đạt ghi 43/43.

7.1 Cách tạo bộ testcase

Script `testcases/tools/generate_testcases.py` sinh toàn bộ 294 đề với seed cố định 20261006. Chạy lại script ra đúng bộ đề cũ, từng ký tự. Script dùng năm cách tạo đề:

- **Lưới đầy ngẫu nhiên.** Quay lui trên lưới trống với thứ tự số xáo trộn. Mỗi lần chạy ra một lưới đầy hợp lệ khác hẳn nhau. Bản đầu của script lấy một lưới mẫu rồi hoán vị hàng, cột và đổi tên số; cách đó chỉ ra một họ lưới nên em bỏ (Chương 6).
- **Đề có lời giải duy nhất.** Từ một lưới đầy, xóa lần lượt từng ô theo thứ tự ngẫu nhiên. Lần xóa nào làm đề có từ hai lời giải trở lên thì trả số lại. Dừng khi đủ số ô trống cần có.
- **Đề nhiều lời giải.** Xóa ngẫu nhiên nhiều ô, không kiểm tra lời giải duy nhất.

- **Đề dựng có chủ đích.** Đề đúng 2 lời giải: xóa 4 ô tạo thành hình chữ nhật mà hai số ở hai góc đối nhau đổi chỗ được. Đề vô nghiệm: lấy một đề có lời giải duy nhất, đổi một số cho sẵn thành số khác mà không tạo số trùng, rồi kiểm tra đề mới không còn lời giải nào. Đề không hợp lệ: thêm số trùng, ký tự lạ, bớt hoặc thêm dòng, ô.
- **Đề nổi tiếng.** Đề minh họa trên Wikipedia, Arto Inkala, AI Escargot, Easter Monster, đề 17 số đầu tiên trong danh sách của Gordon Royle, và đề chống quay lui.

Đề sinh ngẫu nhiên nào cần hơn 1 triệu lần thử thì script bỏ và sinh đề khác, để bộ test chạy nhanh. Lựa chọn này làm các đề ngẫu nhiên nghiêng về phía dễ. Các đề khó thật sự nằm ở nhóm đề nổi tiếng.

Định dạng

Mỗi nhóm là một file văn bản. Mỗi đề trong file là một khối gồm dòng mô tả, phần `input` và phần `expected`. Phần input giữ nguyên từng ký tự, kể cả dòng trống, tab và ký tự xuống dòng kiểu Windows, dùng đúng các định dạng mà hộp Dán đề của website nhận. Phần expected là lưới lời giải, hoặc `NO SOLUTION`, hoặc `INVALID` kèm lý do. File `manifest.csv` ghi thêm cho mỗi đề: nhóm, định dạng, số ô trống, kết quả mong đợi, số lời giải, số lần thử, ghi nhận, trả lại, và số lần bị loại vì hàng, cột, khối.

7.2 Các nhóm testcase

- **Nhỏ (40 đề):** 1 đến 5 ô trống, người đọc tự kiểm bằng tay được. Gồm hai đề mẫu “Dễ nhất” và “Dễ”.
- **Thông thường (150 đề):** 149 đề sinh ngẫu nhiên có lời giải duy nhất, 20 đến 50 ô trống, mỗi mức ô trống khoảng 5 đề. Cộng đề mẫu “Vừa” có 4 lời giải.
- **Lớn, khó (45 đề):** 25 đề có lời giải duy nhất với 51 đến 56 ô trống, 14 đề nhiều lời giải với 58 đến 70 ô trống, và 6 đề nổi tiếng.
- **Biên (13 đề):** lưới đã đầy (0 ô trống), lưới trống (81 ô), chỉ một số cho sẵn (80 ô), một ô trống ở ô đầu và ở ô cuối, trống trọn một hàng, một cột, một khối, đường chéo, ba hàng đầu, và ô trống duy nhất có đáp án là 1 hoặc là 9.
- **Đặc biệt (21 đề):** nhiều lời giải, đúng 2 lời giải, vô nghiệm ngay ô đầu, vô nghiệm phải tìm mới biết; cùng một đề viết theo ba định dạng; dòng trống xen giữa, xuống dòng kiểu Windows, nhiều dấu cách và tab.
- **Không hợp lệ (25 đề):** trùng ở hàng, cột, khối, nhiều chỗ cùng lúc; số 0, chữ cái, số 10, dấu chấm, dấu gạch; thiếu hoặc thừa dòng, ô; chuỗi 80 hoặc 82 ký tự; file rỗng.

Hai đề chậm

Đề chống quay lui cần 622.577.597 lần thử, đề 17 số cần 239.312.393 lần. Trên bộ giải của website, hai đề chạy hết 18 phút 45 giây. Script đánh dấu chúng là đề chậm, không chạy mặc định. Lệnh `yarn testcases --include-slow` chạy cả hai.

7.3 Cách bảo đảm độ phủ

Mỗi đề khi chạy ghi lại những nhánh xử lý của chương trình mà đề đó đi qua. Cả bộ phải đi qua đủ 20 nhánh, thiếu nhánh nào thì script báo lỗi:

- 3 định dạng đề: 9 dòng có dấu cách, 9 dòng viết liền, chuỗi 81 ký tự.
- 5 loại lỗi định dạng: sai số dòng, sai số ô trong một dòng, ký tự lạ trong dòng, chuỗi sai độ dài, ký tự lạ trong chuỗi.
- 3 loại số trùng: ở hàng, ở cột, ở khối.
- 5 kết quả: lưới đã đầy, giải được, giải được nhưng có nhiều lời giải, vô nghiệm ngay (chưa ghi nhận lần nào), vô nghiệm sau khi tìm.
- 3 lý do bị loại: vì hàng, vì cột, vì khối.
- Quay lui sau khi đã thử tới số 9.

Độ phủ theo nhánh chưa đủ để tin bộ test. Em kiểm thêm bằng cách cố tình làm sai bộ giải của website ở bốn chỗ, rồi chạy lại cả bộ (Bảng 7.2). Lần đầu, bộ test chỉ bắt được hai trong bốn lỗi. Sau khi thêm ba cột số lần bị loại theo hàng, cột, khối, và đặt giới hạn số lần thử cho từng đề, bộ test bắt được cả bốn.

Bảng 7.2: Bốn lỗi cố ý và số đề báo sai

Lỗi cố ý	Bộ test phát hiện
Chỉ thử số 1 đến 8, bỏ số 9	229 đề sai, báo thiếu nhánh quay lui sau số 9
Kiểm tra đề bỏ qua khối 3×3	5 đề trùng khối báo sai, báo thiếu nhánh trùng ở khối
Nhận số 0 là ô trống	1 đề sai, đúng đề có số 0
Kiểm tra cột trước hàng	262 đề sai vì số lần bị loại theo hàng, cột lệch

7.4 Đáp án lấy từ đâu

Đáp án do hai chương trình Python độc lập với website tạo ra:

1. **Bản tham chiếu mô hình 1** (`checks/reference_solver.py`). Bản này cài đặt quay lui riêng, cùng thứ tự xét ô và thử số với website. Từ đây có lời giải đầu tiên, số lần thử, ghi nhận, trả lại và số lần bị loại theo hàng, cột, khối.

2. **Bộ đếm lời giải** trong script sinh đề. Bộ đếm dùng cách khác hẳn: chọn ô ít khả năng nhất trước và đánh dấu bằng bitmask. Bộ đếm cho biết đề có bao nhiêu lời giải, tối đa tới 1000.

Script dừng ngay nếu hai chương trình không khớp nhau, hoặc nếu lời giải không đúng luật: 27 hàng, cột, khối phải là hoán vị của 1 đến 9, và các số đề cho phải giữ nguyên. Một kiểm tra chéo nữa đến từ trang Mọi lời giải. Chế độ tìm hết của website đếm lại số lời giải của 251 đề. 239 đề tìm hết trong giới hạn 300 nghìn lần thử, và cả 239 đề khớp bộ đếm độc lập.

Chương 8

Đánh giá

Hệ thống trả về kết quả đúng trên cả 294 đề của bộ testcase. Toàn bộ 723 test đơn vị và 60 test trên Chrome cũng đạt. Các số đo trong chương chạy trên máy Apple M2 Pro, RAM 16 GB, macOS 26.7, Node 24.13 và Chrome 154. Số đo thời gian phụ thuộc máy, còn số lần thử thì không.

8.1 Kết quả trên bộ testcase

Lần chạy ngày 10/10/2026 cho 292 trên 292 đề đạt (Bảng 7.1). Mỗi đề phải khớp đáp án ở bốn chỗ: loại kết quả, nội dung đầu ra, ba bộ đếm, và ba số lần bị loại theo hàng, cột, khối. Cả bộ đi qua đủ 20 nhánh xử lý. Hai đề chậm chạy riêng ngày 6/10 và cũng đạt, nên tổng cộng 294 trên 294 đề đạt.

Kết quả từng đề lưu trong `testcases/results/results.csv`. Lệnh `yarn testcases` chạy lại cả bộ trong khoảng 4 giây.

8.2 Thời gian thực hiện

- **Đề nhỏ và đề thông thường** chạy dưới 30 ms mỗi đề trên bộ giải của website.
- **Đề chậm nhất trong nhóm chạy mặc định** là Easter Monster: 2.357.880 lần thử, 3,07 giây.
- **Hai đề chậm:** đề chống quay lui mất 13 phút 54 giây, đề 17 số mất 4 phút 42 giây.

Phần lớn thời gian đó là cái giá của việc minh họa từng bước. Bộ giải của website là một generator phát sự kiện ở mỗi bước, và mỗi sự kiện đi ngược qua tới 64 tầng đệ quy. Trên đề Arto Inkala, bản đệ quy thường chạy hết 3,7 ms, bản generator mất 623 ms, chậm hơn khoảng 170 lần (đo ngày 3/10). Nút Giải ngay trên trình duyệt chạy khoảng 660 nghìn lần thử mỗi giây, nên đề Inkala xong trong chưa tới 1 giây, còn đề chống quay lui mất khoảng 15 phút. Người xem bấm Dừng được bất cứ lúc nào, trang không treo.

8.3 So sánh các cách giải

Trang So sánh đo 6 cách: 2 thứ tự chọn ô nhân với 3 cách kiểm tra. Bảng 8.1 và Bảng 8.2 là kết quả chạy bằng script `report/tools/measureComparison.ts`. Script dùng đúng hàm đo của trang So sánh: mỗi cách chạy lặp lại tới khi đủ 0,3 giây rồi lấy trung bình.

Bảng 8.1: Số lần thử theo thứ tự chọn ô (ba cách kiểm tra cho cùng một số)

Đề	**Ô trống**	**Điền lần lượt**	**Ô dễ nhất trước**	**Giảm**
Vừa	40	1.909	226	8 lần
Wikipedia	51	37.652	239	158 lần
Arto Inkala	60	445.778	90.665	5 lần
AI Escargot	58	80.491	1.723	47 lần
Easter Monster	60	2.357.880	42.081	56 lần
Chống quay lui	64	622.577.597	407.156	1.529 lần

Bảng 8.2: Thời gian mỗi lần giải, tính bằng mili giây. Lưới: quét lại lưới; Sổ: ba mảng đánh dấu; Công tắc: bitmask

	Điền lần lượt			**Ô dễ nhất trước**		
Đề	Lưới	Sổ	Công tắc	Lưới	Sổ	Công tắc
Vừa	0,034	0,021	0,020	0,141	0,045	0,044
Wikipedia	0,745	0,381	0,370	0,268	0,077	0,068
Arto Inkala	9,36	4,93	4,50	78,7	23,1	20,0
AI Escargot	1,75	0,918	0,823	1,42	0,434	0,369
Easter Monster	53,0	27,4	26,3	33,8	10,4	8,84
Chống quay lui	17.634	8.565	8.887	780	170	163

Bốn điều rút ra từ hai bảng:

- **Cấu trúc dữ liệu chỉ đổi hằng số.** Ba cách kiểm tra cho đúng cùng số lần thử trên mọi đề. Quét lưới chậm khoảng gấp đôi ba mảng đánh dấu: 9,36 ms so với 4,93 ms trên đề Inkala. Bitmask nhanh hơn ba mảng đánh dấu chút ít ở hầu hết các đề, nhưng chậm hơn ở đề chống quay lui (8.887 ms so với 8.565 ms). Đúng như phân tích ở Chương 4: cách kiểm tra chỉ đổi c_2 trong $T(n) = 9\,T(n-1) + c_2$.
- **Thứ tự chọn ô đổi cả cây tìm kiếm.** Điền ô dễ nhất trước giảm số lần thử từ 5 lần (đề Inkala) tới 1.529 lần (đề chống quay lui).
- **Ít lần thử hơn chưa chắc nhanh hơn.** Đề Vừa và đề Inkala đều thử ít hơn khi điền ô dễ nhất trước, mà lại chạy chậm hơn: đề Inkala 23,1 ms so với 4,93 ms. Trước mỗi bước, cách này phải đếm lại khả năng của mọi ô trống còn lại.

- **Khi cây đủ lớn, cách chọn ô thắng hẳn.** Đề chống quay lui giảm từ 8,6 giây xuống 0,17 giây, đề Easter Monster từ 27,4 ms xuống 10,4 ms.

8.4 Kiểm thử tự động

- **723 test đơn vị** (Vitest): bộ giải khớp bản Python từng con số trên các đề mẫu, khớp các thời điểm giữa chừng mà bản thiết kế vẽ lại, ba cách kiểm tra và thứ tự điền ô dễ nhất trước khớp bản Python riêng của chúng, chế độ tìm hết khớp số lời giải, lịch sử xem lại đúng trạng thái khi lùi 1 đến 300 sự kiện, câu thông báo đúng từng ký tự, và toàn bộ 292 đề của bộ testcase.
- **60 test trên Chrome thật** (Playwright): mọi trạng thái trong bản thiết kế, các nút Từng bước, Tự động, Giải ngay, Dừng, lịch sử, cuộc đua và phép đo ở trang So sánh, trang Mọi lời giải, bố cục ở nhiều cỡ màn hình từ 320 đến 2560 px, và không trang nào hiện chỗ chờ điền chưa thay số.
- **Soát code:** TypeScript ở chế độ nghiêm ngặt, oxlint, không còn chữ tiếng Việt nào trong code ngoài file ngôn ngữ.

Em kiểm lại chính bộ test bằng các lỗi cố ý (Bảng 7.2). Hai lần cách này lộ ra lỗ hổng thật, và cả hai lỗ hổng đã vá.

8.5 Tính trực quan của giao diện

Phần minh họa cho người xem thấy cùng lúc bốn thứ: lưới với màu riêng cho từng trạng thái ô, dòng mã giả đang chạy kèm nhãn đúng hoặc sai, vectơ S, và một câu giải thích bằng lời, ví dụ "Ô (4, 7) - thử số 2: hàng 4 đã có số 2, bỏ.". Các chức năng hỗ trợ người học:

- Từng bước, lùi một bước, tự động với tốc độ từ 1 đến 1.000 lần thử mỗi giây, tới lần quay lui kế tiếp, giải ngay.
- Chạy tự động thì dừng ở lần quay lui đầu tiên và hỏi người xem đoán bước tiếp theo.
- Bảng lịch sử 100 bước gần nhất, bấm vào một dòng để xem lại đúng trạng thái lúc đó.
- Cuộc đua hai lưới ở trang So sánh, mỗi ô trống hiện số khả năng còn lại.
- Bộ sưu tập lời giải ở trang Mọi lời giải, tô các ô khác lời giải đầu tiên.

Về khả năng tiếp cận, các cặp màu chữ và nền đạt độ tương phản WCAG AA. Mỗi trạng thái ô có thêm dấu hiệu ngoài màu, ví dụ gạch ngang cho số bị loại và viền nét đứt cho ô vừa trả lại. Lưới đi được bằng phím mũi tên. Trang có chế độ tối, có bố cục riêng cho điện thoại, và hoạt ảnh ở trang chủ đứng yên khi máy bật giảm chuyển động.

Em chưa kiểm thử với người dùng thật. Các góp ý về giao diện và câu chữ đến từ các phiên Claude đóng vai người dùng phổ thông, sinh viên và biên tập viên (Chương 6), cộng với việc em tự dùng thử. Đây là hạn chế lớn nhất của phần đánh giá giao diện.

8.6 Hạn chế và hướng phát triển

Hạn chế

- **Generator chậm.** Để minh họa từng bước, bộ giải trên website chậm hơn bản đệ quy thường khoảng 170 lần. Đề chống quay lui mất khoảng 15 phút khi bấm Giải ngay.
- **Chưa có người dùng thật thử.**
- **Thứ tự điền ô dễ nhất trước** mới có ở trang So sánh, chưa chạy từng bước được ở trang Giải Sudoku.
- **Chưa có lưới cỡ khác** như 4×4 hay 16×16. Em đã lên kế hoạch nhưng dừng lại để dành thời gian cho report.
- **Đề ngẫu nhiên trong bộ testcase nghiêng về phía dễ**, vì script bỏ các đề cần hơn 1 triệu lần thử.
- **Lịch sử chỉ giữ 100 bước gần nhất.** Muốn xem lại bước cũ hơn thì phải chạy lại từ đầu.
- **Website chỉ có tiếng Việt**, dù toàn bộ chữ đã tách sẵn ra một file để dịch.

Hướng phát triển

- Cho trang Giải Sudoku chọn thứ tự điền ô dễ nhất trước và chạy từng bước.
- Chạy Giải ngay bằng bản đệ quy thường trong một luồng riêng của trình duyệt (Web Worker), chỉ dùng generator khi cần xem từng bước.
- Thêm lưới 4×4 và 16×16 để thấy $\mathcal{O}(k^n)$ thay đổi theo k.
- Dùng lại phần minh họa cho các bài quay lui khác trong slide: 8 quân hậu, tổng tập con, tô màu đồ thị.
- Thử với một nhóm người học thật, ghi lại chỗ họ không hiểu.
- Thêm bản tiếng Anh.

Tài liệu tham khảo

[1] Huỳnh Thị Thanh Thương, *CS112_0708. Quay lui (Backtracking), Nhánh cận (Branch and Bound)*, slide bài giảng môn Phân tích và thiết kế thuật toán, Trường Đại học Công nghệ Thông tin, 2026.

[2] A. Levitin, *Introduction to the Design and Analysis of Algorithms*, 3rd ed., Pearson, 2012. Mục 12.1 Backtracking.

[3] P. Norvig, *Solving Every Sudoku Puzzle*, https://norvig.com/sudoku.html.

[4] Wikipedia, *Sudoku solving algorithms*, https://en.wikipedia.org/wiki/Sudoku_solving_algorithms.

[5] Wikipedia, *Sudoku*, https://en.wikipedia.org/wiki/Sudoku.

[6] Wikipedia, *Mathematics of Sudoku*, https://en.wikipedia.org/wiki/Mathematics_of_Sudoku.